On the Feasibility of Quantum Unit Testing

Andriy Miranskyy*, José Campos†, Anila Mjeda‡, Lei Zhang§, Ignacio García Rodríguez de Guzmán¶
*Department of Computer Science, Toronto Metropolitan University, Toronto, Canada
†Faculty of Engineering of University of Porto, Portugal; and LASIGE, Faculdade de Ciências, Universidade de Lisboa, Portugal
‡Department of Computer Science, Munster Technological University, Cork, Ireland
§Department of Information Systems, University of Maryland, Baltimore County, Baltimore County, USA
¶Institute of Technologies and Information Systems, University of Castilla-La Mancha, Spain
avm@torontomu.ca, jcmc@fe.up.pt, anila.mjeda@mtu.ie, leizhang@umbc.edu, ignacio.grodriguez@uclm.es

***Abstract*—The increasing complexity of quantum software presents significant challenges for software verification and validation, particularly in the context of unit testing. This work presents a comprehensive study on quantum-centric unit tests, comparing traditional statistical approaches with tests specifically designed for quantum circuits. These include tests that run only on a classical computer, such as the Statevector test, as well as those executable on quantum hardware, such as the Swap test and the novel Inverse test. Through an empirical study and detailed analysis on 1,796,880 mutated quantum circuits, we investigate (a) each test's ability to detect subtle discrepancies between the expected and actual states of a quantum circuit, and (b) the number of measurements required to achieve high reliability. The results demonstrate that quantum-centric tests, particularly the Statevector test and the Inverse test, provide clear advantages in terms of precision and efficiency, reducing both false positives and false negatives compared to statistical tests. This work contributes to the development of more robust and scalable strategies for testing quantum software, supporting the future adoption of fault-tolerant quantum computers and promoting more reliable practices in quantum software engineering.**

I. Introduction

The increasing maturity of quantum computing technologies has driven the emergence of more sophisticated quantum algorithms and programs. As these programs become more complex, ensuring their correctness and reliability becomes critical, particularly given the non-intuitive nature of quantum mechanics [1]. Traditional software engineering practices, such as unit testing, have been extensively used to validate classical programs [2]. However, the direct adaptation of these techniques to quantum software faces unique challenges [3]–[9], including the exponential growth of the state space and the probabilistic outcomes inherent to quantum measurements.

To address these challenges, researchers have proposed various approaches to validate and verify quantum programs (see [9], [10] for a full review on this topic). The three most popular ones are: the Statistical tests, the Swap test [11], [12], and the Statevector test.

Statistical tests compare the distributions of expected and actual outputs to identify discrepancies. While effective in certain scenarios, statistical tests often require a large number of *measurements* (also known as *shots*) and may produce false positives or negatives, which can hinder practical applicability. The Swap test estimates the overlap between the expected and actual quantum states by using an ancillary qubit and a controlled-SWAP operation. By design, the Swap test does not produce false positives, but can yield false negatives and may require many shots. The Statevector test compares the actual and expected *quantum state vectors* (hereafter abbreviated as *state vectors*) using classical simulation of the quantum circuit, offering precise and accurate results. However, it runs only on a classical computer and scales exponentially with the width of the quantum register.

Consider, as a motivational example, a trivial quantum circuit where a single qubit is initialized in the $|0\rangle$ state and placed into superposition using a Hadamard gate:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix} \approx \begin{bmatrix}0.7071 & 0.7071\\ 0.7071 & -0.7071\end{bmatrix}.$$

Applying this gate to $|0\rangle$ yields the expected state vector:

$$|\phi_{\text{exp}}\rangle = H|0\rangle = \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right]^{\intercal} \approx [0.7071, 0.7071]^{\intercal},$$

where $^{\intercal}$ denotes the transpose. Now, suppose an error is introduced into the Hadamard gate, resulting in a slightly perturbed unitary operation:

$$\hat{H} \approx \begin{bmatrix}0.7066 & 0.7076\\ 0.7076 & -0.7066\end{bmatrix}.$$

The resulting faulty state[1] is therefore:

$$|\phi_{\text{bug}}\rangle = \hat{H}|0\rangle \approx [0.7066, 0.7077]^{\intercal},$$

which deviates from the expected one:

$$|\phi_{\text{bug}}\rangle - |\phi_{\text{exp}}\rangle \approx [-0.0005, 0, 0006]^{\intercal}.$$

On one hand, if we run the fault-free circuit on a noise-free simulator and measure its single-qubit register, we should observe the outcomes 0 and 1 with equal probability of 50.0% each — essentially simulating an unbiased coin flip. On the other hand, if we measure the single-qubit register in the faulty circuit, the probability of observing 0 is $\approx$49.9% and the probability of observing 1 is $\approx$50.1%, indicating a slightly

[1]The root cause of this small deviation could lie at various levels of the software or hardware stack: e.g., a bug in the circuit defined by the programmer, a defect in the gate implementation, an issue in the transpiler, or a hardware fault. However, from the perspective of our example, the specific cause is irrelevant; we simply observe that $|\phi_{\text{exp}}\rangle \neq |\phi_{\text{act}}\rangle$.

TABLE I: Performance of each test in the motivational example.
For each test, we report the number of true positives (TP), number of true negatives (TN), number of false positives (FP), number of false negatives (FN), accuracy (Acc.), precision (Prec.), recall (Rec.), and F_1 score (F_1). MC stands for "Monte Carlo". Best values are reported in **bold**. For all columns, with the exception of FP and FN, higher values indicate better performance.

Test Name	$p < 0.05$								$p < 0.01$							
	TP	TN	FP	FN	Acc.	Prec.	Rec.	F_1	TP	TN	FP	FN	Acc.	Prec.	Rec.	F_1
Number of shots* = 10^5, *as in recent studies on quantum software testing [13]																
χ^2 test	79	127	73	121	0.515	0.520	0.395	0.449	20	182	18	180	0.505	0.526	0.100	0.168
G-test	79	127	73	121	0.515	0.520	0.395	0.449	20	182	18	180	0.505	0.526	0.100	0.168
Multinomial test	**200**	0	200	**0**	0.500	0.500	**1.000**	0.667	**200**	0	200	**0**	0.500	0.500	**1.000**	0.667
MC χ^2 test	63	143	57	137	0.515	0.525	0.315	0.394	15	187	13	185	0.505	0.536	0.075	0.132
MC G-test	64	136	64	136	0.500	0.500	0.320	0.390	16	187	13	184	0.508	0.552	0.080	0.140
MC Multinomial test	64	135	65	136	0.498	0.496	0.320	0.389	17	186	14	183	0.508	0.548	0.085	0.147
Swap test	5	**200**	**0**	195	0.513	**1.000**	0.025	0.049	5	**200**	**0**	195	0.513	**1.000**	0.025	0.049
Statevector test *	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**
Inverse test	9	**200**	**0**	191	0.523	**1.000**	0.045	0.086	9	**200**	**0**	191	0.523	**1.000**	0.045	0.086
Number of shots* = 10^7, *i.e., above the theoretical prediction values based on Equation (8) using the Quantum Chernoff Bound [14], [15]																
χ^2 test	**200**	120	80	**0**	0.800	0.714	**1.000**	0.833	197	189	11	3	0.965	0.947	0.985	0.966
G-test	**200**	120	80	**0**	0.800	0.714	**1.000**	0.833	197	189	11	3	0.965	0.947	0.985	0.966
Multinomial test	**200**	0	200	**0**	0.500	0.500	**1.000**	0.667	**200**	0	200	**0**	0.500	0.500	**1.000**	0.667
MC χ^2 test	**200**	128	72	**0**	0.820	0.735	**1.000**	0.847	196	188	12	4	0.960	0.942	0.980	0.961
MC G-test	**200**	123	77	**0**	0.808	0.722	**1.000**	0.839	196	191	9	4	0.968	0.956	0.980	0.968
MC Multinomial test	**200**	126	74	**0**	0.815	0.730	**1.000**	0.844	197	186	14	3	0.958	0.934	0.985	0.959
Swap test	191	**200**	**0**	9	0.978	**1.000**	0.955	0.977	191	**200**	**0**	9	0.978	**1.000**	0.955	0.977
Statevector test *	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**
Inverse test	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**	**200**	**200**	**0**	**0**	**1.000**	**1.000**	**1.000**	**1.000**

*Note that the Statevector test is independent of the number of shots. It is included in both parts of the table to facilitate comparison.

biased coin flip. Given the small differences in the probabilities of the outcomes in the fault free vs. faulty circuit, we could ask:

Would the previously proposed Statistical tests, the Swap test, or the Statevector test be able to reliably distinguish between the expected and the faulty state?

To answer this question, we ran 200 independent repetitions of six statistical tests.

1) The χ^2 test [16], [17], which has been used in several other works on quantum software testing [13], [18], [19].

Two other that, although powerful, have not been explored in the context of quantum software testing, namely:

2) The G-test [17],
3) The Multinomial test [17].

And their respective Monte Carlo (MC) variants:

4) MC χ^2 test,
5) MC G-test,
6) MC Multinomial test.

Additionally, we also considered the Swap test and the Statevector test. As for the number of shots we set it to 10^5 (as recently suggested by Mendiluze Usandizaga *et al.* [13]) and 10^7 (based on the theoretical estimation[2] of Equation (8) using the Quantum Chernoff Bound [14], [15]). While the latter might be high for practical applications, it provides a conceptual upper bound for analysis. Later in the paper, we explore how few shots are actually necessary for reliable decision-making.

[2]Equation (8) indicates that our Inverse test (later presented in Section III-B4) can distinguish the different between the states with a p-value of 0.05 using using 5.4×10^6 shots, and with a p-value of 0.01 using 8.3×10^6 shots. The chosen value of 10^7 shots accommodates both cases.

Additionally, the statistical tests were configured with two p-values commonly used in the literature: 0.05 (e.g., [18]) and 0.01 (e.g., [13], [19], [20]).

Then, we defined two scenarios:

1) *Negative case*: The actual and expected states are identical accordingly to the test: $|\phi_{\text{act}}\rangle = |\phi_{\text{exp}}\rangle = H|0\rangle$. In other words, there is no fault.
2) *Positive case*: The actual and faulty states differ slightly accordingly to the test: $|\phi_{\text{act}}\rangle = |\phi_{\text{bug}}\rangle = \hat{H}|0\rangle$ and $|\phi_{\text{exp}}\rangle = H|0\rangle$. Here, a fault is present.

And framed the answer to our question as a binary classification problem. When $|\phi_{\text{act}}\rangle = |\phi_{\text{exp}}\rangle$, the ideal outcome is a *true negative* — the test should not report a failure. When the states differ, i.e., $|\phi_{\text{act}}\rangle \neq |\phi_{\text{exp}}\rangle$, the correct outcome is a *true positive* — the test should report a failure. Misclassifications fall into two categories: Type I error, *false positives* (reporting a failure when the states are equal) and Type II error, *false negatives* (failing to report a failure when the states differ). These errors are analogous to flaky tests, i.e., tests that inconsistently pass or fail under the same conditions [21]–[23].

The results are summarized in Table I, and the key observations are as follows.

First, consider the case of 10^5 shots. When the number of shots is significantly lower than the theoretically estimated value, all tests perform poorly. Interestingly, the Swap test and Inverse test occasionally yield correct results by chance, indicating that even a small number of measurements can sometimes succeed. However, while 10^5 may seem large, achieving consistent and reliable results requires more shots. Below, we analyze what happens when the number of shots exceeds the estimated recommended value, specifically 10^7.

Statistical tests produce both false positives and false negatives. False positives are particularly problematic, as they indicate failure even when the code is correct, frustrating for developers. Adjusting the p-value threshold in statistical tests trades off false positives and false negatives. For example, lowering the threshold from 0.05 to 0.01 reduced false positives from 80 to 11 in the χ^2 test, but increased false negatives from 0 to 3. The Multinomial test is highly sensitive and "latches" onto positive cases. This issue is mitigated by using a Monte Carlo version, which samples the expected distribution a number of times[3] to improve robustness. Overall, the Monte Carlo versions of all three statistical tests show slightly better performance in terms of Accuracy, Precision, Recall, and F_1 Score.

The Swap test [11], [12] has been designed to produce no false positives in ideal, noise-free circuits (which is our case), i.e., correctly report no difference when the actual and expected states are equal. However, when the states differ, the test may yield false negatives if the number of shots is insufficient. Although the Swap test reports nine false negatives vs. zero reported by any statistical test, it achieves a higher Accuracy, Precision, and F_1 Score than of the statistical test.

The Statevector test computes and compares the actual and expected state vectors on a classical computer, yielding 100% accuracy and precision, as expected. It is independent of the number of shots because the state vectors are obtained by multiplying the matrix representations of the circuit gates. However, the cost of computing state vectors grows exponentially with the number of qubits.

Our work. To overcome the limitations of the Statistical tests, the Swap test, and the Statevector test, we propose a novel test, the *Inverse test*. It is based on circuit reversion and encodes the expected state behavior directly into the quantum circuit, enabling pass/fail outcomes through measurements. Unlike the statistical tests but similar to the Swap test, the Inverse test operates on a fixed measurement rule: a particular observed output directly indicates test failure. If that output is not observed, more shots may be needed. This binary property makes it conceptually simpler to analyze. Additionally, and similar to the Swap test, the Inverse test is designed to produce no false positives.

Revisiting the motivational example, the Inverse test is on par with the Statevector test, outperforming the Statistical tests and the Swap test across all key performance metrics (Accuracy, Precision, Recall, and F_1 Score). This suggests that the Inverse test has higher statistical power and may require fewer shots to detect subtle differences, making it more computationally efficient. However, *does this advantage persist in other and perhaps more complex quantum circuits?*

To answer this overarching question, we design and conduct an experimental evaluation where the actual output of 1,796,880 quantum circuits intentionally differs from its expected output. We achieve this by mutating 10,000 randomly generated quantum circuits and 85 circuits from the MQT Bench [24]. We then (a) assess how many shots each test requires to reliably detect the discrepancy between each mutant and its original version, (b) analyze tests' false positive and false negative rates, and (c) discuss tests' overall resource efficiency. The results show that the Statevector test and the Inverse test are the strongest contenders.

Our **contributions** are as follows:

- ★ **Formal definition of a quantum unit test**, a test harness inspired by classical software testing, that precisely describes what constitutes a unit test for quantum circuits. (Section III-A)
- ★ **Implementation of Statistical tests, the Swap test, and Statevector test** as a quantum unit test. (Sections III-B1 to III-B3)
- ★ **Definition and implementation of a novel test, the Inverse test**, as a quantum unit test. (Section III-B4)
- ★ **Evidence of tests' efficiency.** We analyze the theoretical time complexity of the quantum unit tests under study, comparing their performance across different setups. (Section III-C)
- ★ **Evidence of tests' effectiveness.** We empirically evaluate across 1,796,880 pairs of original-mutant circuits (2.5 times more mutants than the previous largest empirical study on quantum mutation testing [13], 723,079) and nine quantum unit tests, and show that Statevector test and the Inverse test reliably distinguish between quantum states that are slightly different. (Section IV)

II. Background and Related Work

Quantum assertions vs. Quantum testing. In classical computing, a runtime assertion placed in the program's source code is a predicate used to verify the correctness of a program during its execution. If the assertion passes, the program continues; if it fails, the program typically[4] terminates.

Significant work has been conducted on quantum assertions [25]–[37] and we refer the reader to [9], [10], [29] for a full review. Consider, for example, the statistical assertions [36], that use statistical tests, such as the χ^2 test, to compare multiple measurement outcomes of a qubit versus its expected state. These methods are destructive: once a quantum measurement is performed, the system collapses, and we cannot add further operations to the measured qubit(s). This behavior diverges significantly from the classical understanding of a runtime assertion. For clarity and consistency, in this paper, we refer to such checks as *statistical test* rather than a *statistical assertion* and we encapsulate them within the structure of a quantum unit test (Section III-B1).

On the other hand, consider the Swap test, which involves measuring an auxiliary qubit while leaving the data qubits unchanged (provided that the test passes). This non-destructive behavior allows further operations to be applied to the data qubits after the measurement. As such, the Swap Test more closely aligns with a classical runtime assertion: we can perform a mid-circuit measurement and decide to terminate the program if we measure the "0" value (i.e., assertion passes) on the auxiliary qubit or continue otherwise. Thus, although we use the *Swap test* as a quantum unit test (Section III-B2), it can

[3]We set the number of repetitions in the Monte Carlo statistical test to 1,000; more details in Section IV-C1d.

[4]Runtime assertions are, by default, disabled in Java but enabled in Python, for example.

also be used as a legitimate quantum runtime assertion (unlike the destructive statistical assertions).

Output probability oracle vs. Quantum testing. Statistical tests such χ^2 have been used as oracles to assess whether the distribution of measurement outcomes of a quantum circuit is similar to the expected distribution [13], [18]–[20], [38], [39]. We leverage this knowledge and encapsulate this oracle in a quantum unit test (Section III-B1).

Number of measurements (shots). Qiskit [40], one of the most popular quantum frameworks [41], sets the number of shots to 1,024, by default. Others, in quantum software testing, have used a fixed number of shots in their empirical evaluations. For example, Wang *et al.* [20] used a single shot[5], Ye *et al.* [38] used 10,000 shots[6], and Mendiluze Usandizaga *et al.* [13] used 100,000 shots. To the best of our knowledge, only one study on quantum software testing has explicitly considered multiple shot counts: Pontolillo *et al.* [39] evaluated 12, 25, 50, 100, 200, 400, 800, 1,600, and 3,200 shots.

In contrast, we estimate the number of shots using the Quantum Chernoff Bound approach [14], [15] *for each circuit*, rather than relying on predefined values.

Quantum test case circuit vs. Quantum unit testing. As noted in [9, Fig. 14] and to the best of our knowledge, there is no well-defined definition of what constitutes a quantum-centered unit test for quantum programs. Nevertheless, the closest approach to ours, named QTCC (Quantum Test Case Circuit) [42], formalizes the idea of a quantum test case and provides a pedagogical circuit-based example. As such, it can be seen as a conceptual precursor to our work, proposing a quantum circuit that embeds: (a) the quantum circuit under test, (b) the expected value, and (c) the oracle that verifies the equivalence between the latter and the value obtained when executing the circuit under test.

III. Quantum unit test

In this section, we formalize the notion of a quantum unit test by adapting key principles from classical unit testing to the quantum computing context. Section III-A introduces the formal definition of a quantum unit test, detailing its three essential components: the input, the program under test, and the oracle used to verify correctness. Section III-B follows with concrete implementations of four representative quantum unit tests (Statistical, Swap, Statevector, and Inverse) each structured around the classical Arrange-Act-Assert paradigm [43] and adapted to account for the unique properties of quantum circuits.

A. Formal definition of a quantum unit test

While some concepts of testing software for classical computers apply to quantum software, many others do not [3]–[5]. In the case of unit testing, the core components remain the same [42]: 1) an input, 2) the program under test, and 3) the oracle, i.e., a mechanism to determine whether the actual output matches the expected output. In this section, we present the quantum formalism for these three components, which will support the implementation of quantum unit tests in Section III-B.

1) Input: The *input state* is defined as $|\psi_I\rangle$. Assume this state is described by n qubits, with

$$|\psi_0\rangle = |00\cdots0\rangle = |0\rangle^{\otimes n}$$

as the default starting state in modern quantum computers. Also, assume that the input state can be obtained by applying a unitary operator W to $|\psi_0\rangle$, i.e.,

$$|\psi_I\rangle = W\,|\psi_0\rangle\,.$$

Further assume that W is correctly constructed and free from implementation defects.

When the input state is $|00\cdots0\rangle$, W is an identity matrix. For more complex input states, W must be defined explicitly (e.g., as a state vector or unitary operator) to generate the corresponding gate sequence to transform $|\psi_0\rangle$ into $|\psi_I\rangle$.

If a circuit is partitioned into multiple subroutines and the gates preceding the subroutine under test are known to be correct, the circuit can compute $|\psi_I\rangle$. However, for better modularization and adherence to software engineering practices, generating the state independently (e.g., using a state vector to gate sequence converter) is preferred.

2) Program under test: The program under test is represented by a unitary operator U. It can be the sequence of quantum gates of a complete quantum circuit, a sub-sequence of gates extracted from the quantum circuit, or an actual subroutine in the OpenQASM 3.0 [44] sense. In this paper, we consider the program under test U as the quantum circuit of a quantum program, and we may use the terms quantum circuit or circuit interchangeably, in which case we imply the former.

3) Test oracle:

a) Expected State vs. Actual State: The expected state $|\psi_E\rangle$ is a complete theoretical description of the quantum state that the circuit is intended to produce. It is represented as a state vector containing all the amplitudes (with both magnitude and phase) for every basis state. For example, if we design a circuit to create an entangled state, $|\psi_E\rangle$ captures the exact superposition that should exist. Knowing $|\psi_E\rangle$ allows for a precise, element-by-element comparison against the actual state $|\psi_A\rangle$.

The actual state $|\psi_A\rangle$ represents the outcome of applying the input $|\psi_I\rangle$ to U and it is defined as

$$|\psi_A\rangle = U\,|\psi_I\rangle\,.$$

b) Unknown Expected State: Note that $|\psi_E\rangle$ must be clearly defined (e.g., as a known state vector) in order to verify the correctness of $|\psi_A\rangle$. As in classical unit testing, knowing the expected output is crucial to confirm the program's correctness. A *core limitation* of our approach arises if the expected output for a given program or subroutine is unknown: in that case, one cannot perform a meaningful test. An attempt to alleviate this problem was proposed by Liu *et al.* [45]. They proposed an

[5]Number of shots performed by QUITO [20] https://github.com/Simula-COMPLEX/quito/blob/main/Quito_CoverageRunning/quito.py#L141, accessed July 2025.

[6]Number of shots performed by QuraTest [38] https://github.com/ToolmanInside/quratest/blob/main/oracle.py#L151, accessed July 2025.

approximate test oracle that assesses whether the actual state of a circuit is within a given set of states or a superposition of the states in the set.

That said, this challenge mirrors the age-old problem in classical testing where a test oracle does not always exists to provide a definitive expected output [46]–[49]. In classical software testing, when an expected output is unavailable, empirical methods are often employed. For instance, differential testing can provide insight by executing multiple implementations of the same system, and comparing their results [50]–[53]. In simulation, discrepancies between different implementations may indicate potential errors, allowing for indirect verification. Additionally, in some cases, classically computed probabilities or approximations can serve as a reference to compare against $|\psi_A\rangle$ enabling some level of validation even when the exact expected state is unknown. Equivalent approaches can also be applied or further developed for quantum testing; however, exploring these methods is outside the scope of this paper.

c) Comparison of Expected and Actual States: The test oracle provides a specification of the expected state $|\psi_E\rangle$, representing the correct output of a quantum program for a given input. A separate *mechanism* is then required to compare the actual output state $|\psi_A\rangle$, produced by the circuit under test, against this expected state. If they match (i.e., $|\psi_E\rangle = |\psi_A\rangle$), the test passes; if they differ (i.e., $|\psi_E\rangle \neq |\psi_A\rangle$), the test fails. The next section discusses various methods for performing this comparison.

B. Implementation of quantum unit tests

In this section, we describe, as quantum unit tests, the implementation of the Statistical tests, the modified Swap test, the Statevector test, and the novel Inverse test in Sections III-B1 to III-B4, respectively. To enhance the readability and maintainability of quantum unit tests, each test is structured in three distinct phases as suggested by others for unit tests in the classical realm [43]. The **Arrange** phase is responsible for initializing a quantum testing circuit with an input state ($|00\cdots0\rangle$ by default) and for appending the quantum program under test to the testing circuit. The **Act** phase runs the testing circuit by evolving the quantum state through the unitary operations in the circuit. And finally, the **Assert** phase checks whether the actual states matches the expected state.

1) Statistical tests: Commonly used to measure discrepancies between actual and expected outcomes. The process involves performing S shots and recording the measured bitstrings[7] from the quantum register implementing $|\psi_A\rangle$. The resulting distribution is then compared with the theoretical distribution derived from $|\psi_E\rangle$. Statistical tests, by design, are susceptible to false positives and false negatives errors due to the inherent limitations of frequentist statistics [54, Sec. 8.3]. The likelihood of such errors depends on the specific test, sample size, and the chosen p-value threshold.

A commonly used statistical test in quantum software testing literature [13], [18], [19] is the χ^2 test [16], [17]. To improve robustness and reliability of our study, we also considered

[7]We assume that all n qubits in the quantum register are measured. If only a subset is measured, the same principles apply.

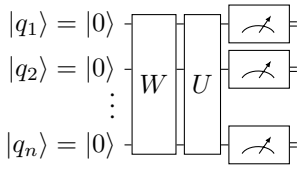

Fig. 1: Quantum testing circuit for the Statistical tests.

Algorithm 1: Statistical test.

Parameter : W as the input state
Parameter : U as the software under test
Parameter : $|\psi_E\rangle$ as the expected output state
Parameter : S as the number of shots
Parameter : p_t as a significance threshold for statistical test
Parameter : *statTest* as the statistical test function, i.e., χ^2 test or G-test
Result: Test result: Pass (U behaves as expected) or Fail (U does not behave as expected)

Arrange

1 Create an empty quantum circuit with a n-qubit register;
2 Append W then U to the circuit;
3 Add measurements to all qubits;
`/* At this point, the circuit looks like the one depicted in Figure 1 */`

Act

4 Run the circuit S times;
5 $V \leftarrow$ Collect measurement outcomes;

Assert

6 $V_E \leftarrow$ Generate expected counts from $|\psi_E\rangle$;
7 $p_s \leftarrow$ *statTest*(V, V_E);
8 **if** $p_s \geq p_t$ **then return** Pass **else return** Fail;

the multinomial test [17], the G-test (a log-likelihood ratio test), as it is often considered more reliable than the χ^2 test in cases with a small number of observations [17], and a Monte Carlo variant of the χ^2 test, Multinomial test, and G-test. The quantum test outlined in Algorithm 1 implements the χ^2 test, G-test, and Multinomial test; and Algorithm 2 the Monte Carlo variants of the three statistical tests. Both start by creating an empty quantum testing circuit with n-qubit register and then append the program's input state (W) the the program under test (U) (lines 1 and 2). Measurements are then added to all qubits and the testing circuit is executed S times (lines 3 and 4). Once the testing circuit is finished, measurements are collected and compared against the expected ones (lines 6 to 8 in Algorithm 1 and lines 6 to 17 in Algorithm 2, respectively).

The number of shots S required for these tests depends on the proximity of the states $|\psi_A\rangle$ and $|\psi_E\rangle$. The closer the states are, the harder they are to distinguish, and thus a larger S is needed. Question is, *what is the minimum number of shots S for a statistical test to be effective?* This is a nontrivial question [55]. To ensure reliable results, Cochran's rule [56, p. 420] suggests that each bitstring should have at least 5 expected observations. Additionally, Pearson's [16] original recommendation requires a minimum of 13 total observations.

Algorithm 2: Monte Carlo version of Statistical test.

Parameter : W as the input state
Parameter : U as the software under test
Parameter : $|\psi_E\rangle$ as the expected output state
Parameter : S as the number of shots
Parameter : p_t as a significance threshold for statistical test
Parameter : *statTest* as the statistical test function, e.g., χ^2
Parameter : M as a number of Monte Carlo repetitions
Result: Test result: Pass (U behaves as expected) or Fail (U does not behave as expected)

Arrange

Create an empty quantum circuit with a n-qubit register;
Append W then U to the circuit;
Add measurements to all qubits;
`/* At this point, the circuit looks like the one depicted in Figure 1 */`

Act

Run the circuit S times;
$V \leftarrow$ Collect measurement outcomes;

Assert

$V_E \leftarrow$ Generate expected counts from $|\psi_E\rangle$;
$s_e \leftarrow$ *statTest*(V, V_E);
$c \leftarrow 0$; `// Counter for Monte Carlo samples` $\geq s_e$
for 1 **to** M **do**
  $V_S \leftarrow$ Sample S synthetic measurements from $|\psi_E\rangle$ using multinomial distribution;
  $s_s \leftarrow$ *statTest*(V_S, V_E);
  if $s_e \geq s_s$ **then**
    $c \leftarrow c + 1$;
  end
end
$p_e \leftarrow c/M$; `// Compute empirical` p-value
if $p_e \geq p_t$ **then return** Pass **else return** Fail;

2) Swap test: Introduced by [11], [12], measures the similarity between the actual state $|\psi_A\rangle$ and the expected state $|\psi_E\rangle$ by estimating the square of their inner product $|\langle\psi_A|\psi_E\rangle|^2$. This test is implemented using a quantum circuit as shown in Figure 2, that prepares the two states and applies n controlled-SWAP gates on them (one for each pair of qubits), controlled by an auxiliary qubit q_a initialized to $|0\rangle$. These controlled-SWAP gates are sandwiched between two Hadamard gates applied to the q_a.

If the states are orthogonal, the inner product $|\langle\psi_A|\psi_E\rangle|^2 = 0$; if they are identical, $|\langle\psi_A|\psi_E\rangle|^2 = 1$. Thus, as shown in [12, p. 167902-2], if the two states are identical, the auxiliary qubit (q_a) will always be measured as 0 (i.e., with probability $P = 1$); if the states are orthogonal, the probability of measuring q_a as 0 drops to 0.5:

$$P(M_{q_a} = 0) = \frac{1}{2} + \frac{1}{2}\left|\langle\psi_A|\psi_E\rangle\right|^2, \tag{1}$$

where M_{q_a} denotes measurement on qubit q_a.

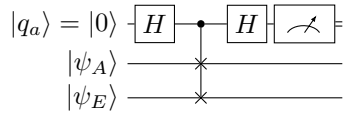

Fig. 2: Quantum testing circuit for the Swap test.

Algorithm 3: Swap test.

Parameter : W as the input state
Parameter : U as the software under test
Parameter : $|\psi_E\rangle$ as the expected output state
Parameter : S as the number of shots
Result: Test result: Pass (U behaves as expected) or Fail (U does not behave as expected)

Arrange

Create an empty quantum circuit with a n-qubit register;
Append W then U to the circuit;
Append $|\psi_E\rangle$ to the circuit;
Append Hadamard and SWAP gates to the circuit as in Figure 2;

Act

Run the circuit S times;
$V \leftarrow$ Collect measurement outcomes;

Assert

if V *contains only zeros* **then return** Pass **else return** Fail;

In typical applications, the Swap test runs multiple times to estimate the similarity between the states (computed by aggregating the values from the measurements) [11], [12]. However, for the purposes of unit testing, we do not need to compute an estimate of the inner product. Instead, if any measurement returns 1, we conclude that the states differ and the test case has failed.

The implementation of the Swap test is provided in Algorithm 3. In practice it is often more efficient to perform S measurements upfront and search the resulting vector for a non-zero value, although one can perform one measurement at a time and stop if a value of 1 is measured. We adopt the former approach.

The Swap test is designed to never produce a false positive, as per Equation (1): q_a will always be 0 if the states are equal. However, note that the test can produce a false negative if the value of S is small. As shown in Equation (1), the closer the states are, the lower the probability of outcome of $q_a = 1$. In our empirical evaluation (see Section IV) we observed that the Swap test does not produce false positives but can yield false negatives.

3) Statevector test: On a classical computer, a simulated (yet accurate) state vector $|\psi_A\rangle$ of a quantum circuit can be computed by sequentially multiplying the unitary matrices that represent its gates[8]. This provides direct access to every amplitude in the quantum state, allowing one to compare the

[8]For example, this can be done using `Statevector.from_instruction(circuit)` in the Qiskit framework [57].

Algorithm 4: Statevector test.

Parameter : W as the input state
Parameter : U as the software under test
Parameter : $|\psi_E\rangle$ as the expected output state
Result: Test result: Pass (U behaves as expected) or Fail (U does not behave as expected)

Arrange

1 Create an empty quantum circuit with a n-qubit register;
2 Append W then U to the circuit;

Act

3 $|\psi_A\rangle \leftarrow$ Compute the final state vector of the circuit;

Assert

4 **if** $|\psi_A\rangle \approx |\psi_E\rangle$ **then return** Pass **else return** Fail;

simulated state $|\psi_A\rangle$ with an expected state $|\psi_E\rangle$.[9]

Since the simulation yields the complete quantum state, one can verify the circuit's correctness at a granular level, ensuring not only correct output probabilities but also accurate coherence and phase relationships. However, this approach becomes computationally infeasible for moderate numbers of qubits, as the state vector grows exponentially with size 2^n.

Despite this limitation, a Statevector test remains a valuable test. Many quantum programs under test involve circuits small enough that the classical simulation is still practical. In such cases, this test is highly reliable, avoiding both false negatives and false positives (within numerical tolerance)[10].

Algorithm 4 outlines the Statevector test. This test offers high accuracy and low run-time, as it requires only a single simulation rather than multiple executions or shots, though it is limited to small quantum registers, as previously mentioned. The algorithm constructs a test circuit by initializing it with W, then appending the program under test U (as shown in Figure 1, excluding the measurements). Then it computes the resulting state vector $|\psi_A\rangle$ and compares it with the state vector of $|\psi_E\rangle$. If the vectors match (within tolerance), the test passes, indicating that U behaves as expected. Notably, in our empirical evaluation (Section IV), the Statevector test successfully detected all faulty states in the faulty circuits considered.

4) Inverse test: Although statistical tests (Section III-B1) only require n qubits to construct the circuit, they involve error-prone test cases. The Swap test (Section III-B2) effectively eliminates false positives but requires expanding the quantum register from n to $2n+1$ qubits. The Statevector test (Section III-B3) produces perfect results but lacked scalability. This raises the question: *can we design a quantum-centric unit test that only uses n qubits, avoids false positives, and scales better than the Statevector test?* The Inverse test, described below, is our attempt to answer this question.

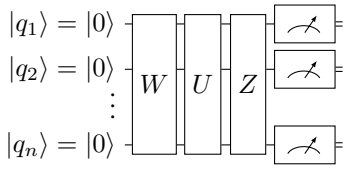

Fig. 3: Quantum testing circuit for the Inverse test.

Algorithm 5: Inverse test.

Parameter : W as the input state
Parameter : U as the software under test
Parameter : $|\psi_E\rangle$ as the expected output state
Parameter : S as the number of shots
Result: Test result: Pass (U behaves as expected) or Fail (U does not behave as expected)

Arrange

1 Create an empty quantum circuit with a n-qubit register;
2 Append W then U to the circuit;
3 Append the conjugate transpose (inverse) of $|\psi_E\rangle$ as Z to the circuit;
4 Add final measurements to all qubits in the circuit;
`/* At this point, the circuit looks like the one depicted in Figure 3 */`

Act

5 Transpile the circuit for a backend (either a simulator or an actual device);
6 Run the transpile circuit S times;
7 $V \leftarrow$ Collect measurement outcomes;

Assert

8 **if** V *contains only zero-bitstrings, i.e.,* $(00\cdots0)$ **then return** Pass **else return** Fail;

The core idea is to revert[11] the output state $|\psi_A\rangle$ back to $|00\cdots0\rangle$, effectively reducing the circuit output to a single deterministic value, namely, the zero-bitstring. This is achieved by constructing a unitary operator Z such that:

$$|\psi_R\rangle = |00\cdots0\rangle \longleftarrow Z\,|\psi_A\rangle\,.$$

Obviously, if U is implemented correctly, then $|00\cdots0\rangle \longleftarrow W^\dagger U^\dagger\,|\psi_A\rangle$, where $\dagger$ superscript denotes the conjugate transpose. While $W^\dagger$ can be computed from W using frameworks like Cirq [65], PennyLane [66], or Qiskit [57], computing conjugate transpose of U directly would not reveal defects in U. Instead, Z is constructed using the conjugate transpose of the expected state $|\psi_E\rangle$, ensuring that $|\psi_R\rangle = |00\cdots0\rangle$,

[9] Comparing expected and actual state vectors to assess the correctness of a program in a simulator is a common technique among practitioners. Although we could not find a formal academic reference, this approach frequently appears on platforms like StackOverflow, where users mention comparing state vectors to verify the correctness of a program. Here is a typical example: "For a given unitary, I want to know whether this unitary gate is correctly evolved in the circuit. In the simulator, I can use 'statevector' to get the state vector to check the correctness of the evolution." [58]

From a theoretical perspective, we conjecture that this method is rooted in the concept of *trace distance* in quantum information theory, defined as $\frac{1}{2}\,\mathrm{Tr}\,|\rho-\sigma|$, where one compares the density matrices of two states ρ and σ to assess their similarity (see [1, Sec. 9.2.1] for details). Since a density matrix ρ can be computed from a pure state vector $|\psi\rangle$ using $\rho = |\psi\rangle\langle\psi|$, this may have been the origin of the comparison.

[10] In this paper, we used a tighter tolerance value of 1×10^{-10}. Note this value is lower than the default value of 1×10^{-8} defined in numpy's `np.allclose` function and in Qiskit's `equiv` function, because the comparison of identical state vectors in our context requires high precision.

[11] We are inspired by prior works that explore the reversibility of circuits, e.g., [30], [59]–[64], conducted in different contexts and complementary to our approach.

only if U is correct. This works because quantum operations are reversible [1]: applying the complex conjugate of a unitary operation to its result restores the original state. After constructing the testing circuit, we can perform repeated measurements[12] to assess its result. Unlike the Statevector test (Section III-B3), this test uses actual circuit measurements, making it more scalable for complex programs.

Algorithm 5 outlines the Inverse test. It constructs a test circuit, initializes it with W and appends the program under test U, constructing $|\psi_A\rangle$. Then, it appends conjugate transpose (inverse) of $|\psi_E\rangle$. The circuit is then transpiled for a given backend (simulator or hardware), run for multiple shots, and the output measurements V are collected. If all observed bitstrings are zero, the test *passes*.

As for the Statistical tests and the Swap test, the question is, *how many shots S are needed to confidently conclude that non-zero string is never measured?* This depends on the closeness of the faulty state to the correct state. The nearer the two states are, the more shots are required. Specifically, the number of required shots grows exponentially as the faulty state approaches the correct state[13]. We formally analyze this exponential growth with the help of the Quantum Chernoff Bound, denoted ξ_{QCB} (see [14], [15] for further details). As per [14, Eqs. 1 and 2],

$$\begin{aligned} P_e &\sim \exp(-N\xi_{\text{QCB}}), \quad \text{where} \\ \xi_{\text{QCB}} &= \lim_{N\to\infty} -\frac{\ln P_e}{N} \\ &= -\ln \min_{0\le s\le 1} \operatorname{Tr}\left(\rho^s \sigma^{1-s}\right) \end{aligned} \tag{2}$$

and P_e is the error probability, N is the number of shots, Tr denotes the matrix trace, ρ is the density matrix representing a passing test case (the zero-state), and σ is the density matrix for the buggy state that differs from the zero-state.

From Equation (2), given ρ and σ, we can compute the required number of shots N for a specified error probability P_e as:

$$N \sim \frac{\ln(P_e)}{\ln \min_{0\le s\le 1} [\operatorname{Tr}(\rho^s \sigma^{1-s})]}. \tag{3}$$

a) Numerator Analysis of Equation (3): Technically, $P_e \in (0,1)$, but in practice a tester would usually use $0 < P_e \le 0.05$. As P_e decreases, the absolute value of the numerator $\ln(P_e)$ increases.

b) Denominator Analysis of Equation (3): The density matrix ρ for a passing test case (zero-state) is given by:

$$\rho = |0\rangle^{\otimes n} \langle 0|^{\otimes n} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}.$$

Matrix ρ contains $2^n \times 2^n$ elements, with only one non-zero element (ρ_{11}). Since $\rho^s = \rho$ for any s, Equation (3) simplifies to:

$$N \sim \frac{\ln(P_e)}{\ln \min_{0\le s\le 1} [\operatorname{Tr}(\rho\sigma^{1-s})]}. \tag{4}$$

However, the structure of σ can vary significantly, making the computation of N challenging for arbitrary cases. Below, we analyze specific cases of σ to build intuition about the behavior of the denominator.

(a) Diagonal Pure States

Case 1: σ is diagonal with $\sigma_{11} = 0$. Consider a pure state where the density matrix σ is diagonal, i.e., $\sigma_{ij} = 0$ for all $i \neq j$, and $\sigma_{11} = 0$. The remaining diagonal elements sum to 1:

$$\sigma = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ 0 & \sigma_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{mm} \end{bmatrix}, \quad \text{with } \sum_{i=2}^{m} \sigma_{ii} = 1,$$

where $m = 2^n$. In this case:

$$\operatorname{Tr}(\rho\sigma^{1-s}) = 1.$$

Substituting into Equation (2) yields:

$$P_e \sim \exp(-N \cdot 0) = 1. \tag{5}$$

This indicates that for trivial cases where ρ and σ are orthogonal, a single shot suffices to differentiate them.

Case 2: σ is diagonal with $0 < \sigma_{11} < 1$. Now consider σ where $0 < \sigma_{11} < 1$ and some other diagonal elements are non-zero, such that $\operatorname{Tr}(\sigma) = 1$. In this case:

$$\operatorname{Tr}(\rho\sigma^{1-s}) = \sigma_{11}^{1-s}.$$

To minimize $f(\sigma_{11}, s) = \ln(\sigma_{11}^{1-s})$, we compute its derivative and solve it for zero:

$$\frac{\partial f(\sigma_{11}, s)}{\partial s} = -\ln(\sigma_{11})^{1-s} \ln[\ln(\sigma_{11})] = 0.$$

Since $\sigma_{11} \in (0,1)$, there are no values of s that satisfy this equality, i.e., we do not have a critical point when $s \in [0,1]$. Thus, the minimum must occur at one of the boundaries of the domain for s:

- At $s = 0$: $f(\sigma_{11}, 0) = \ln(\sigma_{11})^1 = \ln(\sigma_{11})$,
- At $s = 1$: $f(\sigma_{11}, 1) = \ln(\sigma_{11})^0 = 1$.

As $\sigma_{11} \in (0,1)$, $\ln(\sigma_{11}) < 0$ and the minimum is reached when $s = 0$. Substituting $\ln(\sigma_{11})$ into Equation (4) yields:

$$N \sim \frac{\ln(P_e)}{\ln \min_{0\le s\le 1} [\operatorname{Tr}(\rho\sigma^{1-s})]} = \frac{\ln(P_e)}{\ln(\sigma_{11})}. \tag{6}$$

In practice, we round up N to the nearest integer away from zero, ensuring there is at least one shot:

$$N \sim \max\left\{ \left\lceil \frac{\ln(P_e)}{\ln(\sigma_{11})} \right\rceil, 1 \right\}, \tag{7}$$

where $\lceil \cdot \rceil$ denotes the ceiling function. Figure 4 shows how N in Equation (7) changes with σ_{11} and P_e. As $\sigma_{11} \to 1$, N increases exponentially. Conversely, as σ_{11} decreases, N

[12] If necessary, the measurement of $|00\cdots0\rangle$ can be reduced to $|0\rangle$ using one additional gate and an auxiliary qubit. After appending the Z operator in Figure 3, add a multi-controlled gate with zero-controls on $q_1, q_2, \ldots, q_n$. This gate flips the auxiliary qubit only if all n control qubits are in the $|0\rangle$ state. As a result, the auxiliary qubit measures $|1\rangle$ if the test passes, and $|0\rangle$ if it fails.

[13] As an analogy, consider two coins: one fair ($p = 0.5$ for heads), and one slightly biased ($p = 0.5 + 10^{-10}$). Distinguishing them reliably would require a very large number of flips.

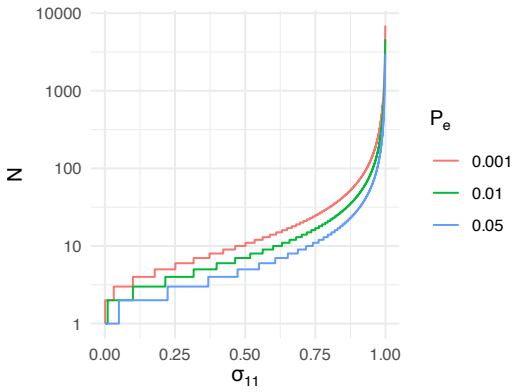

Fig. 4: The value of N from Equation (7) for $0.001 \leq \sigma_{11} \leq 0.999$ and $P_e \in \{0.001, 0.01, 0.05\}$.

quickly approaches 1. The value of P_e also contributes to N, which increases as P_e decreases. However, σ_{11} has a stronger influence on N compared to P_e.

(b) Non-Diagonal Pure States and Mixed States

When non-diagonal elements are present in σ, symbolic analysis becomes challenging and often requires numerical methods. The complexity increases further with mixed states[14]. However, the same principle holds: $N \to \infty$ as $\sigma \to \rho$. Intuitively, as the two states become more similar, more measurements are needed to differentiate them. However, if σ is very close but not identical to ρ, this approximate similarity may be sufficient for certain practical use cases where perfect discrimination is unnecessary. This is analogous to scientific computing, where computations often involve approximations and numerical errors due to factors such as finite-precision arithmetic. In such contexts, it is generally acceptable to tolerate a small deviation from the exact solution, provided that the error remains within a pre-specified tolerance level. Similarly, the near-equivalence of σ and ρ can still yield meaningful and reliable results when the application does not require absolute precision.

To develop an intuition for the shot count S, which is defined using the same principles as Equation (7), we express it as

$$S \sim \max\left\{\left\lceil \frac{\ln(P_e)}{\ln \min_{0\leq s\leq 1}\left[\mathrm{Tr}\left(\rho\sigma^{1-s}\right)\right]}\right\rceil, 1\right\}. \tag{8}$$

S varies for an arbitrary state σ; thus, it is useful to analyze the behavior of S on a range of empirical circuits (which we perform in Section IV). This is needed because S gives us an approximation rather than the exact shot count.

In summary, our analysis shows that a single shot suffices to distinguish orthogonal states, while distinguishing nearly identical states may require $> 10^7$ shots. We confirm this behavior empirically in Section IV.

C. Analysis of complexity

Table II compares the complexity of the four quantum-centric unit tests. We focus on a common practical case in which the fault is detected, making the circuits for $|\psi_A\rangle$ and $|\psi_E\rangle$ nearly identical in terms of depth and gate count.

1) Circuit Width: Among all methods, the Swap test requires the widest circuit, using $2n+1$ qubits, i.e., roughly twice as many as the others. Since execution costs grow exponentially with the qubit count, this makes the Swap test the most expensive to simulate and potentially more difficult to execute on early fault-tolerant quantum devices, where the number of available qubits will be limited.

2) Circuit Depth and Gate Count: The Inverse test requires the deepest circuit (approximately twice the depth of the other methods) leading to longer execution times on a quantum device. Both the Swap test and Inverse test have roughly twice the gate count compared to the Statistical tests and Statevector test.

3) Shot Count: Statistical tests require multiple shots. Although the Swap test and Inverse test also require repeated execution, they might potentially require fewer shots. In contrast, the Statevector test is deterministic and does not require any quantum execution or simulation, i.e., it can be computed purely through classical matrix multiplications.

4) False Positives/Negatives: The Statevector test is the most reliable, producing neither false positives nor false negatives. Statistical tests are the least reliable, susceptible to both types of errors. The Swap test and Inverse test lie in between — they avoid false positives but can still yield false negatives.

5) Classical Compute Time (Preparation):

a) When $|\psi_A\rangle$ is a circuit and $|\psi_E\rangle$ is a state vector: If the state vector $|\psi_E\rangle$ is dense, all methods incur exponential cost due to the size of the vector (2^n). For the statistical tests, this corresponds to sampling from a distribution over 2^n elements. In the Statevector test, $|\psi_E\rangle$ is directly available, but it requires handling $2^n \times 2^n$ matrix multiplications representing $|\psi_A\rangle$. For the Swap test and Inverse test, converting the classical state vector $|\psi_E\rangle$ into a quantum circuit requires circuit synthesis, which also incurs exponential cost [1, Sec.4.5.4]; see [67] for an overview of conversion techniques.

However, if $|\psi_E\rangle$ is sparse with only r non-zero elements ($2^n \gg r$), the statistical tests requires only $O(r)$ operations to sample from the distribution. The cost of the Statevector test remains unchanged, as the representation of $|\psi_A\rangle$ is unaffected. For the Swap test and the Inverse test, the cost of synthesizing a circuit for $|\psi_E\rangle$ drops to $O(nr)$; see [67] for a review of conversion methods. In this case, the statistical tests has the lowest computational complexity, followed by the Inverse test, the Swap test, and finally the Statevector test.

b) When both $|\psi_A\rangle$ and $|\psi_E\rangle$ are circuits: This scenario can occur during post-optimization validation. For instance, suppose we have an existing circuit, $|\psi_E\rangle$. After optimizing it, we obtain a new version, $|\psi_A\rangle$, and we want to ensure that the optimized circuit still produces the correct state.

Classical preparation time for the statistical tests and the Statevector test remains unchanged. For the Swap test and Inverse test, prep time drops from exponential to linear, since the circuit for $|\psi_E\rangle^\dagger$ can be obtained by inverting the original circuit (which is linear in gate count).

In this case, the Inverse test becomes the most efficient, followed by the Swap test, particularly on fault-tolerant quantum

[14]Mixed state σ is generally harder to distinguish from ρ because they may share partial support in the same subspace, leading to a higher shot count.

TABLE II: Comparison of the non-quantum-centric and quantum-centric unit tests described in Section III-B.
Functions $D(x)$ and $G(x)$ denote the circuit depth and gate count, respectively, for implementing unitary operators or states x. Useful relations include: $D(|\psi_A\rangle) = D(UW)$, $D(|\psi_E\rangle) = D(|\psi_E\rangle^\dagger) = D(Z) = D(Z^\dagger)$, $G(|\psi_A\rangle) = G(U) + G(W)$, and $G(|\psi_E\rangle) = G(|\psi_E\rangle^\dagger) = G(Z) = G(Z^\dagger)$. The approximations shown, denoted by $\approx$, hold only if the circuits for $|\psi_A\rangle$ and $|\psi_E\rangle$ are nearly identical in terms of depth and gate count.

	Statistical tests	**Swap test**	**Statevector test**	**Inverse test**
Circuit width	n	$2n+1$	n	n
Circuit depth	$D(\|\psi_a\rangle)$	$\max(D(\|\psi_a\rangle), D(\|\psi_E\rangle)) + n + 2 \approx D(\|\psi_a\rangle) + n$	$D(\|\psi_a\rangle)$	$D(\|\psi_R\rangle) \approx 2D(\|\psi_a\rangle)$
Circuit number of gates	$G(\|\psi_A\rangle)$	$G(\|\psi_A\rangle) + G(\|\psi_E\rangle) + n + 2 \approx 2(G(\|\psi_A\rangle)) + n$	$G(\|\psi_A\rangle)$	$G(\|\psi_A\rangle) + G(\|\psi_E\rangle) \approx 2(G(\|\psi_A\rangle))$
Shot count (S)	≥ 13	≥ 1	0	≥ 1
Can produce false-positives?	Yes	No	No	No
Can produce false-negatives?	Yes	Yes	No	Yes
Classical computer prep. time when $\|\psi_A\rangle$ is a circuit and $\|\psi_E\rangle$ is not	$O(G(\|\psi_A\rangle) + S(\|\psi_E\rangle))$ *	$O(G(\|\psi_A\rangle) + C(\|\psi_E\rangle) + n)$ §	$O(2^n)$	$O(G(\|\psi_A\rangle) + C(\|\psi_E\rangle))$ §
Classical computer prep. time when $\|\psi_A\rangle$ and $\|\psi_E\rangle$ are circuits	$O(G(\|\psi_A\rangle) + 2^n)$ *	$O(G(\|\psi_A\rangle) + G(\|\psi_E\rangle) + n)$	$O(2^n)$	$O(G(\|\psi_A\rangle) + G(\|\psi_E\rangle^\dagger))$
Classical computer simulator time	$O(2^n)$	$O(2^{2n+1})$	$O(2^n)$	$O(2^n)$
Quantum computer time	$O(D(\|\psi_a\rangle))$	$O(\max(D(\|\psi_a\rangle), D(\|\psi_E\rangle))) \approx O(D(\|\psi_a\rangle))$	0	$O(D(\|\psi_R\rangle)) \approx O(D(\|\psi_a\rangle))$ ‡

*If the state vector for $|\psi_E\rangle$ is dense, sampling the vector V_E (see Algorithm 1) requires $O(2^n)$ operations. Thus, the computational complexity for sampling $S(|\psi_E\rangle)$ is $O(2^n)$. However, if the state vector is sparse, the complexity drops to $S(|\psi_E\rangle) = O(r)$, where r is the number of nonzero elements in $|\psi_E\rangle$ state vector. This significantly reduces preparation time.
‡ Quantum computer time for the Inverse test will be twice that of the Statistical and Swap test, assuming the circuits for $|\psi_A\rangle$ and $|\psi_E\rangle$ are similar.
§ If $|\psi_E\rangle$ is a dense state vector, converting it to a circuit has computational complexity $C(|\psi_E\rangle) = O(2^n)$. For a sparse state vector, the complexity reduces to $C(|\psi_E\rangle) = O(nr)$, where r is the number of nonzero elements in $|\psi_E\rangle$ statevector.

hardware. While simulation costs still grow exponentially, the preparation and quantum execution times become manageable, making it suitable for larger circuits.

6) Simulator and Quantum Hardware Execution: The Swap test remains the most resource-intensive to simulate due to its circuit width. In contrast, the Statistical tests, the Statevector test, and the Inverse test operate on n-qubit registers, resulting in lower simulation costs. On quantum hardware, Statistical tests are the fastest, followed by the Swap test. The Inverse test requires circuits that are twice as deep. However, for the all-circuit case, discussed above, it is the most practical option for fault-tolerant devices, provided that $|\psi_A\rangle$ and $|\psi_E\rangle$ can be prepared efficiently on a classical computer and testers aim to avoid false positives.

7) Summary: When n is small, the Statevector test executed on a classical computer yields the most reliable results, with exact answer and no false positives or negatives. If large n cannot be avoided and the test has to be executed on a fault-tolerant quantum computer, the Inverse test is the best option (for a dense state vector $|\psi_E\rangle$). If n is large and $|\psi_E\rangle$ is a sparse state vector, then Statistical tests are the cheapest option (although the programmer will suffer from false positives and negatives). The Inverse test and the Swap test are more expensive alternatives in this case. However, while they require more classical computation time (and in the case of the Swap test, twice as many qubits) they eliminate concerns about false positives.

IV. Empirical Study

This paper aims to evaluate several quantum unit tests on the following research questions (RQs):

RQ1: Which testing method performs best at reliably distinguishing between quantum states that are different?

RQ2: Which testing method requires the fewest shots to detect true positives while keeping false negatives low?

A. Defects

In this work, we focus on defects in the program code written by the developer, assuming that the rest of the stack is free of defects. To simulate defects one might introduce, we applied four mutation operators, one at time, to a quantum circuit, either randomly-generated or real. Three mutation operators (i.e., QGR or simply Replace, QGD or Remove, and QGI or Add) have been proposed [68], [69] and we, in this study, propose a new one (i.e., RGI). Each operator is presented below. Note that Fortunato *et al.* [68] has also proposed mutation operators QMI and QMD but neither were not included in our study as both operators would only generate equivalent mutants. The former operator introduces measurements at any point in the circuit, which would then be removed by our experimental scripts before computing circuit's state vector (e.g., in the Statevector test) as it requires a measurement-free circuit. In other words, once the all measurements are removed from the original and the mutated circuits, the circuits are equivalent. The latter, which deletes measurements from the circuit, would also be pointless. Suppose there is a circuit with four measurements. The QMD operator would randomly select a measurement and delete it. Our experimental scripts, would then remove all measurements in the original circuit (i.e., four) and all the remaining measurements in the mutated circuits (i.e., three). Thus, both circuits would be equivalent.

Quantum Gate Replacement (QGR) operator replaces an existing quantum gate call in the circuit (e.g., `circuit.x(qubit)`) with each of its syntactically equivalent gates[15] (e.g., `circuit.h(qubit)`, `circuit.s(qubit)`, etc.), generating a mutant per replacement.

Quantum Gate Deletion (QGD) mimics an error where a programmer accidentally removes a gate from the circuit.

Quantum Gate Insertion (QGI) performs the opposite of QGD and inserts an additional call to a syntactically equivalent quantum gate immediately after an existing gate call (e.g., adding `circuit.y(qubit)` after `circuit.x(qubit))`, creating mutants for each possible equivalent gate.

[15]A syntactically equivalent quantum gate, as defined by Fortunato *et al.* [68], is a gate that can replace another in code without causing syntax errors, even if it performs a different quantum operation.

Rotation Gate Insertion (RGI) simulates an error where a programmer introduces a rotational gate[16]:

$$\text{RGate} = \exp\left[-i\frac{\theta}{2}(\cos\phi x + \sin\phi y)\right],$$

at an arbitrary position in the circuit, with a small rotation deviation of $\theta = \phi = \frac{\pi}{180}$ (i.e., one-degree angle).

B. Subjects

We apply mutation operators described in Section IV-A to two datasets of quantum circuits discussed below. To conserve computational resources, since memory and computation costs grow exponentially with the number of qubits n, we capped n at 5 for both datasets.

a) MQT Bench: The Munich Quantum Toolkit Benchmark Library (MQT Bench)[17] [24] v1.1.9 is composed of 1,943 quantum programs (implementing various quantum algorithms) of different sizes. Restricting our analysis to $n = 5$ qubits yields 85 circuits with a number of qubits spans between 2 and 5 and circuit depth varies between 1 and 403 instructions. We then mutate each circuit. Due to time constraints, for each circuit and mutation operator, we only consider 10% (selected at random) of all possible mutants. This generates a total of 45,030 mutated circuits.

b) Randomly-generated circuits: To increase the number and diversity of the quantum circuits used in our empirical study, we randomly generate circuits using Qiskit's `random_circuit`[18] module. We randomly generate 1,000 *different* circuits for all combinations of number of qubits 1, 2, 3, 4, and 5; and depth 1 and 10. This results in 10,000 circuits which we then mutate to generate a total of 1,751,850 mutated circuits.

Note, *no mutated circuit* in either the MQT Bench or the Randomly-generated circuits dataset *is equivalent to its correspondent original circuit*. Specifically, we removed any pair for which the original and mutated circuits produced identical statevectors (within a numerical tolerance of 10^{-10}). There were 2,189 $\leftarrow$ 47,219 $-$ 45,030 equivalent pairs in the MQT Bench and none in the Randomly-generated circuits dataset (as non-equivalent pairs were generated by design). This filtering ensures that all mutants in our evaluation represent semantically distinct behaviors. As a result, we do not have any negative cases in our study.

Also note that while our filtering removes equivalent *original-mutant* pairs, it is possible that *two different mutants* of the same circuit produce identical state vectors. We do not account for such cases in this work, but acknowledge this as an avenue for future investigation.

C. Experimental approach

1) Setup:

a) Circuits' inputs: The quantum circuits are initialized, in each quantum unit test, with the default input state, i.e., $|00\cdots0\rangle$.

b) Shot count: The maximum number of shots is set to 10^6 for the MQT Bench and to 10^4 for the Random Circuits dataset. For individual experiments (i.e., a test on a pair original-mutant), the number of shots is capped at twice the theoretical limit given by Equation (8) with $P_e = 0.05$. This cap is applied across all tests, although the analysis focuses specifically on the Inverse test. Since our goal is to identify the top-performing test method rather than to conduct an in-depth analysis of each method, this approach is sufficient.

c) Repetitions: To increase reproducibility, each experiment (i.e., a test on a pair original-mutant) is repeated 100 times, each time with a different seed value, as the sequence of obtained measurements varies with each execution.[19] Thus, the total number of experiments is therefore $1,585,665,000 = (1,751,850 + 10,000)$ pairs $\times$ 9 tests $\times$ 100 repetitions.

d) Number of repetitions in the Monte Carlo statistical tests: To conserve computational resources, we set the number of Monte Carlo repetitions M in Algorithm 2 to 1000. Given $P_e = 0.05$, we use a consistent p-value threshold of $p_t = 0.05$ for the Monte Carlo tests. The standard error (SE) of the Monte Carlo estimate, assuming a binomial distribution, is then

$$\text{SE} = \sqrt{\frac{p_t(1-p_t)}{M}} = \sqrt{\frac{0.05 \cdot 0.95}{1000}} \approx 0.007.$$

e) Runtime environment: The experiments were carried out on a high performance computing cluster equipped with Intel Xeon Gold 6148 Skylake and AMD EPYC 9654 Zen 4 CPUs. Each experiment was allocated 16 GB of memory and 2 CPU cores. The total computational cost for the "production" run was approximately 9.3 CPU-years. The code was executed using Python v.3.11.5 with packages qiskit v.1.4.0, qiskit-aer v.0.16.1, scipy v1.16.0, and numpy v2.3.1.

2) Metrics:

a) True Positive (TP), False Negative (FN), and Recall: For each test, we report the number of true positives, the number of false negatives, and the recall. Since these experiments are designed without negative cases, related metrics such as accuracy and precision are omitted.

b) Determining a number of shots for a given observation: To determine the number of shots for a given for the Swap and Inverse tests, we simply identify the first non-zero measurement string in the vector of measured values.

For the statistical test, our aim is to find the minimum number of shots required for the p-value to fall below a specified threshold $p_t = 0.05$. Typically, p-values start high with low shot counts and decrease as the shot count increases, though not always monotonically. A naive approach would increase the shot count from 1 until the p-value drops below the threshold, but this is computationally expensive. Instead, we use a two-pass search strategy: 1) Perform a coarse scan to identify a range of shot counts where the p-value crosses

[16] RGate documentation: https://docs.quantum.ibm.com/api/qiskit/1.3/qiskit.circuit.library.RGate, accessed July 2025.

[17] MQT Bench homepage: https://www.cda.cit.tum.de/mqtbench, accessed July 2025.

[18] Qiskit Random Circuits documentation: https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit_random, accessed July 2025.

[19] The recommendations in classical software engineering is to run experiments that might have non deterministic results at least 30 repetitions [70]. We chose a higher value to further increase the confidence in our results.

the threshold; 2) Use binary search within that range to find the exact minimum shot count.

c) Ranking tests: When comparing the number of shots, we use a dense rank approach. For example, if four test methods require 3, 7, 8, and 7 shots, respectively, their ranks would be 1, 2, 3, and 2.

3) Procedure: For each pair of expected and actual circuits (that is, expected and mutated circuits) we compare their state vectors to evaluate the Statevector test. This comparison is done once per pair.

For the remaining tests, we construct the circuits shown in Figures 1 to 3 and perform the specified number of measurements for each, as outlined in paragraph IV-C1b. We then post-process the results to determine whether each test passed or failed (following the algorithms described in Section III-B). This entire process is repeated 100 times to increase robustness, as the observed measurement sequences vary with each run.

D. Results and Answers to RQs

As shown in Figure 5, while many experiments require fewer than 10^3 shots to reliably distinguish between quantum states that are different, some require up to 10^5 shots. The distribution also has heavy tails; for example, some cases require more than 10^{15} shots to detect a difference. Both datasets show similar distribution profiles, which is expected given the use of comparable mutation operators that produce similar differences between the actual and expected states.

Figure 7 shows that, for the MQT Bench dataset, the **Inverse test requires the fewest shots (rank-1) in ≈69% of cases**, followed by the Swap test at ≈33%. The multinomial test ranks third with ≈17%, but results in more false negatives (and is known to be prone to false positives as we saw in Section I), making it potentially risky to use. The remaining statistical tests show similar behavior but perform worse overall.

For the Random Circuits dataset in Figure 7, a similar trend is observed. The **Inverse test performs best, achieving rank-1 results in ≈78% of cases**, followed by the Swap test at ≈40%. The statistical tests perform similarly to each other, with rank-1 results below 1%.

Figure 6 illustrates the change in the percentage of rank-1 outcomes across all tests and datasets. No clear trend emerges with respect to the number of qubits; however, additional datasets will be needed to determine whether this pattern is generalizable.

Answer to RQ1: The Inverse test performs best at reliably distinguishing between quantum states that are different.

To evaluate differences in shot count, Figure 8 provides useful insights. For the MQT Bench dataset, the **Inverse test requires significantly fewer shots**: the median is below 10 for both the Inverse test and the Swap test, with the former requiring slightly fewer shots (2 vs. 5). In contrast, the statistical tests have a median shot count ≈1900 (with the exception of the multinomial test with ≈600).

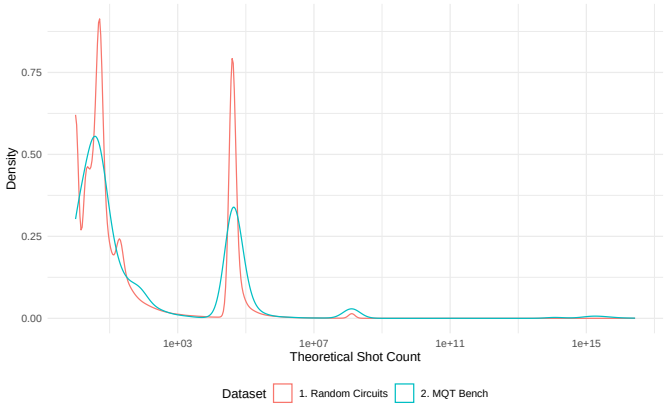

Fig. 5: Theoretical shot count S based on Equation (8) with $P_e = 0.05$.

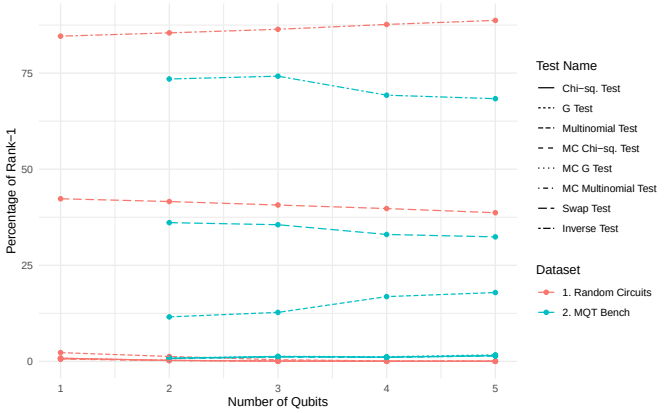

Fig. 6: Variation in the percentage of rank-1 cases with the number of qubits.

For the Random Circuits dataset in Figure 8, the difference is also pronounced: the median **number of shots for the Inverse test is 3 and for the Swap test is 9**. The statistical tests have a median between approximately 600 and 700 shots.

We do not include outliers in Figure 8 due to the large number of observations, which makes plotting them computationally expensive. The whiskers of the box-and-whisker plot indicate that the 1.5 interquartile range is relatively small: less than 10^4 for Random Circuits and less than 10^5 for the MQT Bench. However, it is important to note that some cases exceeded these values. In fact, the maximum number of shots observed in both datasets reached the upper limit allowed: approximately 10^5 for Random Circuits and 10^7 for the MQT bench. Thus, in practice, a large number of shots may still be required, especially when the extent of difference between the faulty and correct circuits is unknown.

Table III further supports these findings. By construction, the Statevector test are independent of shot count and achieves perfect recall. Among the remaining tests, recall is generally lower for the Random Circuits dataset than for the MQT Bench dataset. This is expected due to the difference in shot counts (10^5 vs. 10^7), highlighting the importance of having sufficient data for reliable testing. Notably, the relative drop in recall is

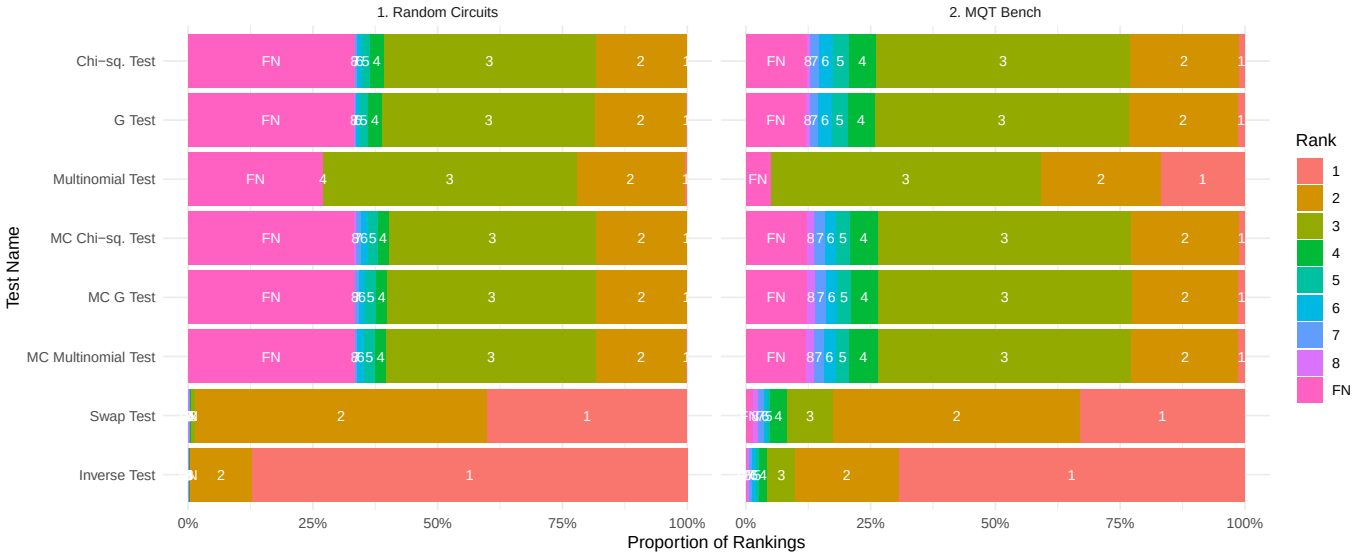

Fig. 7: Ranking performance comparison.

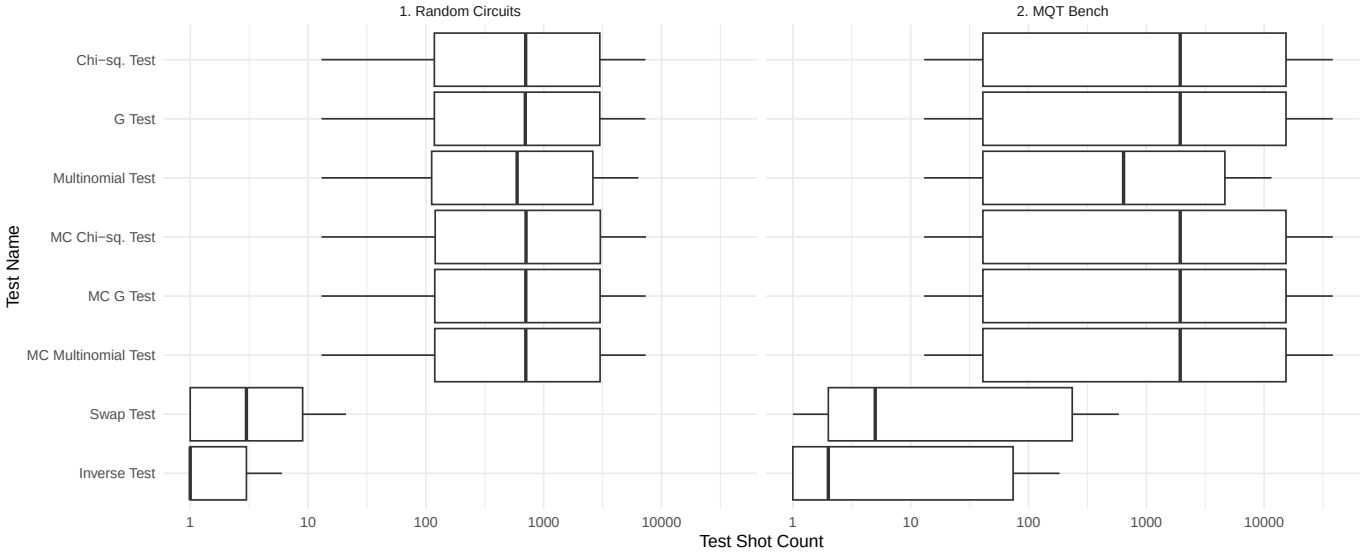

Fig. 8: Shot count performance comparison (outliers not visualized). False negatives are excluded from the plots.

TABLE III: Performance of each tests in our empirical study. Top-3 best values are reported in **bold**.

Test Name	Random Circuits dataset			MQT Bench dataset		
	TP	FN	Recall	TP	FN	Recall
χ^2 test	9.77E+07	7.75E+07	0.558	3.87E+06	6.28E+05	0.860
G-test	9.79E+07	7.73E+07	0.559	3.88E+06	6.25E+05	0.861
Multinomial test	1.25E+08	5.04E+07	0.712	4.26E+06	2.40E+05	0.947
MC χ^2 test	9.74E+07	7.78E+07	0.556	3.87E+06	6.32E+05	0.860
MC G-test	9.76E+07	7.76E+07	0.557	3.87E+06	6.32E+05	0.860
MC Multinomial test	9.75E+07	7.77E+07	0.557	3.87E+06	6.32E+05	0.860
Swap test	1.44E+08	3.09E+07	**0.824**	4.31E+06	1.89E+05	**0.958**
Statevector test	1.75E+08	0.00E+00	**1.000**	4.50E+06	0.00E+00	**1.000**
Inverse test	1.58E+08	1.68E+07	**0.904**	4.39E+06	1.14E+05	**0.975**

smallest for the Inverse test ($0.071 \leftarrow 0.975 - 0.904$), followed by the Swap test ($0.134 \leftarrow 0.958 - 0.824$), and then the statistical tests (e.g., the χ^2 test shows a drop of $0.302 \leftarrow 0.860 - 0.558$).

For Random Circuits dataset, **the Inverse test achieves the highest recall** (0.904), followed by the Swap test test (0.824) and then the statistical tests (approximately 0.56 for all, except the multinomial test, which is prone to "latching" and reaches 0.712). For the MQT Bench dataset, the **Inverse test and Swap test have similar recall values** (0.975 **and** 0.958**, respectively**), while most statistical tests perform lower (around 0.86), with the multinomial test achieving a higher recall of 0.95.

Overall, **the results suggest that quantum-centric tests (Inverse and Swap) provide more robust performance**. As a side note, our analysis indicates that about 3.3% of cases expected to be flagged were missed, slightly below the anticipated 5% (as $P_e = 0.05$). This implies that the theoretical formula for required shot count from Equation (8), aligns well with empirical results and may even be slightly conservative in practice.

Answer to RQ2: In terms of the median, the Inverse test requires 60% to 66% less shots than the second best, the Swap test, to detect true positives while keeping false negatives low.

E. Threats to validity

We classify validity threats as suggested by Yin [71] and Wohlin *et al.* [72].

1) Internal validity: We developed our own codebase, which may introduce implementation errors. To reduce this risk, we wrote unit tests, conducted code reviews, and followed standard practices like version control and documentation to ensure correctness and traceability.

2) Construct validity: To ensure validity, we evaluated each test under controlled conditions with known ground truth and modeled outcomes as binary classification problems, following standard practice in empirical software testing [72]–[74]. We used established statistical baselines and explicitly defined quantum unit test procedures to minimize ambiguity.

3) External validity: Throughout the previous sections, we assume execution of the tests on a Fault-Tolerant (FT) quantum computer or an idealized, noise-free simulator. Although the same programs can be executed on Noisy Intermediate-Scale Quantum (NISQ) hardware using error mitigation techniques [75], [76], our focus remains on FT devices or simulators. This decision is motivated by the projection that FT quantum computers, expected to become available around 2029 (e.g., IBM promises a computer able to execute "100 million quantum gates on 200 logical qubits [77]"), are the most likely to deliver quantum advantage and generate practical business value [78]. Thus, prioritizing the FT backend is both a forward-looking and a pragmatic approach.

Software engineering studies often face challenges due to the variability of real-world environments, and the issue of generalization remains inherently difficult to resolve [79]. Further threats to external validity stem from our design choices.

First, the quantum programs evaluated in this study are quantum algorithms which may or may not reflect the diversity of real-world quantum applications, potentially limiting generalizability. To minimize this threat we also considered 10,000 randomly generated quantum circuits.

Second, the fault model is based on the predefined set of mutation operators proposed by Fortunato *et al.* [68], [80], [81] and Mendiluze *et al.* [69][20] and a percentage of the generated mutants. These constraints may bias the fault set toward simpler or faster-to-generate cases, underrepresenting faults that occur in practice. However, the lack of large datasets of quantum faults that have occurred in real quantum software makes it impossible to truly assess this. QBugs [82] is not available. The Bugs4Q [83] dataset of *bugs* in Qiskit only contains 20 *bugs* (out of 42) related to the source code of a quantum program, and most could be mimicked by the set of mutation operators we used in our study. The remaining ones are, for example, related to refactors[21], related to the draw of the circuit[22], or related to the execution of the circuit[23]; which are outside the scope of this paper.

Third, regarding the pairs programs-mutants, our set is similar to the one used by Mendiluze Usandizaga *et al.* [13], for a number of qubits up to 5. We considered 85 circuits from the MQT Bench and they considered 80 (the difference is due to the usage of a more recent version of the MQT Bench). For the 85 circuits we generated 45,030 mutants vs. 45,067 (some either missed or with compilation issues)[24].

Finally, our evaluation uses a fixed set of statistical and p-value thresholds (i.e., 0.05 and 0.01). Although our evaluation considers the statistical test commonly used in quantum software testing [13], [18], [19], i.e., the χ^2 test, we also consider five other powerful tests. Nevertheless, different statistical tests or parameterizations could yield different outcomes. These limitations highlight the need for future work exploring a broader corpus of quantum programs, fault models, and evaluation baselines.

Our theoretical complexity analysis is dataset-agnostic, which supports broad generalizability. However, some tactical questions (such as the relationship between test ranks and the number of qubits discussed in Section IV-D) may require further investigation. This opens opportunities for other researchers to replicate our work on alternative datasets and assess its generalization to different quantum software systems.

V. Discussion

In this section we briefly describe the novelty of our work, the three main takeaways, and finally discuss a practical approach to design a testable quantum circuit.

A. Novelty

- ★ The statistical tests and the Statevector test we use follow established approaches commonly used in both the literature and practice.
- ★ The Swap test is well-known; we have made a minor modification to simplify classical post-processing.
- ★ As for the Inverse test, the underlying concept (using a reversible circuit [30], [59]–[64]) is standard in quantum programming. For example, it is commonly applied in NISQ devices for noise assessment. However, to the best of our knowledge, our formulation is novel in the context of quantum unit tests.
- ★ Regarding the efficiency and effectiveness of these tests, we also have not seen any comparison.

[20] Main difference between Fortunato *et al.* [68], [80], [81]'s mutation operators and Mendiluze *et al.* [69]'s is that the former only inserts or replaces a gate to a syntactically equivalent one rather to any gate. Although a more restrict approach, Fortunato *et al.*'s do not generate invalid mutants by design.

[21] Bugs4Q #5: https://github.com/Z-928/Bugs4Q-Framework/blob/main/qiskit/5/modify_5.txt, accessed July 2025.

[22] Bugs4Q #22: https://github.com/Z-928/Bugs4Q-Framework/blob/main/qiskit/22/modify_22.txt, accessed July 2025.

[23] Bugs4Q #4: https://github.com/Z-928/Bugs4Q-Framework/blob/main/qiskit/4/modify_4.txt, accessed July 2025.

[24] https://github.com/EnautMendi/Quantum-Circuit-Mutants-Empirical-Evaluation/issues/2, accessed July 2025.

B. Takeaways

1) By construction, both quantum-centric tests, the Inverse and Swap tests, yield zero false positives (see Sections III-B2 and III-B4).
2) The Inverse test performs best, followed closely by the Swap test, at reliably distinguishing between quantum states that are slightly different (see Section IV-D). The Statevector test also shows excellent results, with zero false positives and false negatives; however, it is not scalable (see Section III-B3).
3) In theory, classical resource requirements depend on the setup and how the actual and expected states are represented (see Section III-C). In practice, the Inverse test requires the fewest shots (potentially translating to lower quantum compute time), followed by the Swap test (see Section IV-D). While statistical tests are generally the cheapest in terms of raw compute cost, their long-term expense may be higher due to the overhead of handling false positives and false negatives, which often demands significant human effort.

C. Designing Testable Quantum Circuits

To ensure that the software is testable, we can design Python functions that generate quantum circuits in a scalable manner, based on an input parameter n (e.g., consider the `QFT` Python class[25] in Qiskit [85], which accepts `num_qubits` as an input parameter).

These functions dynamically adjust the circuit's structure according to n. The implementation typically involves control flow constructs, such as loops and conditional branches, that depend on n. Therefore, we can apply standard software testing practices, such as path coverage (where every code path is executed at least once) and control structure testing, which includes simple and nested loops.

Which values of n should we choose for testing? Here, we can apply the *equivalence partitioning* approach to select representative values. The partitions depend on the subroutine being tested, but the following cases are commonly relevant.

- For $n = 1$, the circuit may consist only of single-qubit gates (although this is not always the case).
- For $n = 2$, two-qubit gates (e.g., CX) may be introduced.
- For $n = 3$, multi-controlled gates like the Toffoli (C^2X, where 2 represents the number of control qubits) gate may appear.
- For $n \geq 4$, more general multi-controlled gate ($C^{n-1}X$) may be required.

While we can create tests for large values of n, this may conceptually contradict the nature of a unit test, where we aim to construct simple, isolated inputs for which it is easy to compute the expected output. The primary goal of unit testing is to cover potential execution paths within the subroutine under test while maintaining clarity and feasibility in verifying expected results. Therefore, we suggest designing subroutines in a way that all execution paths can be covered, if possible, by testing the inputs for small values of n.

[25]This class generates a circuit for the Quantum Fourier Transform [84].

Example: Suppose that we aim to test Grover's algorithm [86] and our $n = 128$, a size that cannot be simulated classically (as the state vector has 2^{128} elements). Assuming that the implementation of the algorithm follows the principles described above, we can create smaller versions of circuits for $n = 1, 2, 3, 4$ (using various values of W). This approach enables reasonably thorough testing of the code in a simulator while keeping computational costs manageable.

VI. Conclusions and Future Work

Conclusion: In this paper, we present an evaluation of four representative test groups: Statistical, Statevector, Swap, and the novel Inverse test. The evaluation is based on controlled experiments conducted on both synthetic and benchmark quantum circuits.

Our findings indicate that, although statistical tests remain standard in many contexts, their destructive nature and reliance on high shot counts limit their practicality. The Statevector test, which requires complete state information, is infeasible on real quantum hardware. The Swap test, which operates on an auxiliary qubit without altering the data qubits state (if the test passes), produced low false positive rates in our experiments but showed lower recall than the Inverse test.

The proposed Inverse test demonstrates high statistical power, no false positives, and strong recall across all tested datasets. It also requires lower number of shots than the other tests in many cases. While its worst-case circuit preparation complexity is theoretically exponential, this is often not a limiting factor in practice. As a result, the Inverse test offers a computationally efficient and effective alternative. In light of our findings, we encourage the community working on fault detection in quantum circuits to adopt the Inverse test.

Future work: Future work will further investigate the generalizability of these findings, beginning with input state variation. Specifically, we plan to extend our evaluation to circuits initialized with a broader range of inputs, including automatically generated states, as explored in prior work [18]–[20], [38], [87], [88]. We also aim to scale testing to larger and noisier circuits, integrate these methods into hybrid quantum-classical workflows, and develop composable testing toolkits to support practical quantum software engineering.

Acknowledgments

The authors thank profusely Shaukat Ali and Paolo Arcaini for their insightful discussions and valuable contributions to this research. Our heartfelt thanks also go to the organizers of the Dagstuhl Seminar 24512[26] namely, Shaukat Ali, Johanna Barzen, Andrea Delgado, Hausi A. Müller, and Juan Manuel Murillo, for bringing us together and catalyzing our collaboration.

This work was partially supported by the Natural Sciences and Engineering Research Council of Canada (grant # RGPIN-2022-03886), the LASIGE Research Unit, ref. UID/00408/2025 - LASIGE, the QCloud QuantumEd project funded by the

[26]Dagstuhl Seminar 24512 – Quantum Software Engineering homepage: https://www.dagstuhl.de/en/seminars/seminar-calendar/seminar-details/24512, accessed July 2025.

EOSC *INFRAEOSC-03-2020* (grant #101017536), CyberSkills HCI Pillar 3 Project 18364682, Science Foundation Ireland grant 13/RC/2094_P2, and Q-SERV-Q&T Project (PID2021-124054OB-C32, of the Ministry of Economy, Industry and Competitiveness and FEDER). The authors thank the Digital Research Alliance of Canada for providing computational resources.

References

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge Univ. Press, 2010. DOI: 10.1017/CBO9780511976667.

[2] E. Daka and G. Fraser, "A survey on unit testing practices and problems," in *2014 IEEE 25th International Symposium on Software Reliability Engineering*, IEEE, 2014, pp. 201–211.

[3] A. Miranskyy and L. Zhang, "On testing quantum programs," in *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2019, pp. 57–60. DOI: 10.1109/ICSE-NIER.2019.00023.

[4] A. Miranskyy, L. Zhang, and J. Doliskani, "Is your quantum program bug-free?" In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER '20, Association for Computing Machinery, 2020, 29–32. DOI: 10.1145/3377816.3381731.

[5] A. Miranskyy, L. Zhang, and J. Doliskani, *On testing and debugging quantum software*, 2021. DOI: 10.48550/arXiv.2103.09172. arXiv: 2103.09172.

[6] M. Piattini, M. Serrano, R. Perez-Castillo, G. Petersen, and J. L. Hevia, "Toward a quantum software engineering," *IT Professional*, vol. 23, no. 1, pp. 62–66, 2021. DOI: 10.1109/MITP.2020.3019522.

[7] M. Piattini *et al.*, "The talavera manifesto for quantum software engineering and programming," in *Short Papers Proceedings of the 1st International Workshop on the QuANtum SoftWare Engineering & pRogramming, Talavera de la Reina, Spain, February 11-12, 2020*, M. Piattini, G. Peterssen, R. Pérez-Castillo, J. L. Hevia, and M. A. Serrano, Eds., ser. CEUR Workshop Proceedings, vol. 2561, CEUR-WS.org, 2020, pp. 1–5. [Online]. Available: https://ceur-ws.org/Vol-2561/paper0.pdf.

[8] J. Zhao, *Quantum software engineering: Landscapes and horizons*, 2021. arXiv: 2007.07047 [cs.SE]. [Online]. Available: https://arxiv.org/abs/2007.07047.

[9] N. C. L. Ramalho, H. Amario de Souza, and M. Lordello Chaim, "Testing and debugging quantum programs: The road to 2030," *ACM Trans. Softw. Eng. Methodol.*, Jan. 2025, ISSN: 1049-331X. DOI: 10.1145/3715106. [Online]. Available: https://doi.org/10.1145/3715106.

[10] J. M. Murillo *et al.*, "Quantum software engineering: Roadmap and challenges ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. DOI: 10.1145/3712002.

[11] A. Barenco, A. Berthiaume, D. Deutsch, A. Ekert, R. Jozsa, and C. Macchiavello, "Stabilization of quantum computations by symmetrization," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1541–1557, 1997. DOI: 10.1137/S0097539796302452.

[12] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, "Quantum fingerprinting," *Phys. Rev. Lett.*, vol. 87, p. 167 902, 16 2001. DOI: 10.1103/PhysRevLett.87.167902.

[13] E. n. Mendiluze Usandizaga, S. Ali, T. Yue, and P. Arcaini, "Quantum circuit mutants: Empirical analysis and recommendations," *Empirical Software Engineering*, vol. 30, no. 4, Apr. 2025, ISSN: 1382-3256. DOI: 10.1007/s10664-025-10643-z.

[14] K. M. R. Audenaert, J. Calsamiglia, R. Muñoz-Tapia, E. Bagan, L. Masanes, A. Acin, and F. Verstraete, "Discriminating states: The quantum chernoff bound," *Phys. Rev. Lett.*, vol. 98, p. 160 501, 16 Apr. 2007. DOI: 10.1103/PhysRevLett.98.160501.

[15] M. Nussbaum and A. Szkoła, "The Chernoff lower bound for symmetric quantum hypothesis testing," *The Annals of Statistics*, vol. 37, no. 2, pp. 1040–1057, 2009. DOI: 10.1214/08-AOS593.

[16] K. Pearson, "X. on the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 50, no. 302, pp. 157–175, 1900. DOI: 10.1080/14786440009463897.

[17] J. H. McDonald, *Handbook of biological statistics*, 3rd ed. Sparky House Publishing, 2014.

[18] X. Wang, P. Arcaini, T. Yue, and S. Ali, "QuSBT: Search-based testing of quantum programs," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 173–177.

[19] X. Wang, P. Arcaini, T. Yue, and S. Ali, "QuCAT: A combinatorial testing tool for quantum software," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2023, pp. 2066–2069.

[20] X. Wang, P. Arcaini, T. Yue, and S. Ali, "Quito: A coverage-guided test generator for quantum programs," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 1237–1241.

[21] L. Zhang, M. Radnejad, and A. Miranskyy, "Identifying flakiness in quantum programs," in *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM*, IEEE, 2023, pp. 1–7. DOI: 10.1109/ESEM56168.2023.10304850.

[22] L. Zhang and A. Miranskyy, "Automated flakiness detection in quantum software bug reports," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 2, 2024, pp. 179–181. DOI: 10.1109/QCE60285.2024.10274.

[23] J. Sivaloganathan, A. Jamshidi, A. Miranskyy, and L. Zhang, "Automating quantum software maintenance: Flakiness detection and root cause analysis," *arXiv*, 2024. DOI: 10.48550/arXiv.2410.23578.

[24] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, 2023, MQT Bench is available at https://www.cda.cit.tum.de/mqtbench/.

[25] Y. Huang and M. Martonosi, "Statistical assertions for validating patterns and finding bugs in quantum programs," in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA '19, Phoenix, Arizona: Association for Computing Machinery, 2019, pp. 541–553, ISBN: 9781450366694. DOI: 10.1145/3307650.3322213.

[26] H. Witharana, D. Volya, and P. Mishra, "Quassert: Automatic generation of quantum assertions," *arXiv preprint arXiv:2303.01487*, 2023.

[27] H. Zhou and G. T. Byrd, "Quantum circuits for dynamic runtime assertions in quantum computation," *IEEE Computer Architecture Letters*, vol. 18, no. 2, pp. 111–114, 2019.

[28] J. Liu, G. T. Byrd, and H. Zhou, "Quantum circuits for dynamic runtime assertions in quantum computation," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 1017–1030. DOI: 10.1109/LCA.2019.2935049.

[29] P. Li, J. Liu, Y. Li, and H. Zhou, "Exploiting quantum assertions for error mitigation and quantum program debugging," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*, IEEE, 2022, pp. 124–131.

[30] T. Patel and D. Tiwari, "Qraft: reverse your Quantum circuit and know the correct program output," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21, Virtual, USA: Association for Computing Machinery, 2021, 443–455, ISBN: 9781450383172. DOI: 10.1145/3445814.3446743.

[31] T.-F. Chen, C.-Y. Wei, and J.-H. R. Jiang, "Vanqira: A vanishing-state-based framework for quantum circuit runtime assertion," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, IEEE, vol. 1, 2023, pp. 1033–1043.

[32] I. G.-R. de Guzmán, A. G. de la Barrera Amo, M. Á. Serrano, M. Polo, and M. Piattini, "Quantumunit: A proposal for classic multi-qubit assertion development," in *Conference on Cloud Computing, Big Data & Emerging Topics*, Springer, 2024, pp. 121–131.

[33] D. Rovara, L. Burgholzer, and R. Wille, "Automatically refining assertions for efficient debugging of quantum programs," *arXiv preprint arXiv:2412.14252*, 2024.

[34] D. Rovara, L. Burgholzer, and R. Wille, "A framework for debugging quantum programs," *arXiv preprint arXiv:2412.12269*, 2024.

[35] G. Li, L. Zhou, N. Yu, Y. Ding, M. Ying, and Y. Xie, "Projection-based runtime assertions for testing and debugging quantum programs," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020. DOI: 10.1145/3428218.

[36] Y. Huang and M. Martonosi, "Statistical assertions for validating patterns and finding bugs in quantum programs," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 541–553.

[37] J. Liu and H. Zhou, "Systematic approaches for precise and approximate quantum state runtime assertion," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2021, pp. 179–193.

[38] J. Ye, S. Xia, F. Zhang, P. Arcaini, L. Ma, J. Zhao, and F. Ishikawa, "QuraTest: Integrating Quantum Specific Features in Quantum Program Testing," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '23, Echternach, Luxembourg: IEEE Press, 2024, 1149–1161, ISBN: 9798350329964. DOI: 10.1109/ASE56229.2023.00196.

[39] G. Pontolillo, M. R. Mousavi, and M. Grzesiuk, *QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit*, 2025. arXiv: 2503.22641 [quant-ph].

[40] G. Aleksandrowicz *et al.*, *Qiskit: An Open-source Framework for Quantum Computing*, version 0.7.2, Jan. 2019. DOI: 10.5281/zenodo.2562111. [Online]. Available: https://doi.org/10.5281/zenodo.2562111.

[41] F. Ferreira and J. Campos, "An exploratory study on the usage of quantum programming languages," *Science of Computer Programming*, vol. 240, p. 103 217, 2025, ISSN: 0167-6423. DOI: https://doi.org/10.1016/j.scico.2024.103217.

[42] A. García de la Barrera Amo, M. A. Serrano, I. García Rodríguez de Guzmán, M. Polo, and M. Piattini, "Automatic generation of test circuits for the verification of quantum deterministic algorithms," in *Proceedings of the 1st International Workshop on Quantum Programming for Software Engineering*, ser. QP4SE 2022, Singapore, Singapore: Association for Computing Machinery, 2022, 1–6, ISBN: 9781450394581. DOI: 10.1145/3549036.3562055.

[43] C. Wei, L. Xiao, T. Yu, S. Wong, and A. Clune, "How Do Developers Structure Unit Test Cases? An Empirical Analysis of the AAA Pattern in Open Source Projects," *IEEE Trans. Softw. Eng.*, vol. 51, no. 4, 1007–1038, Jan. 2025, ISSN: 0098-5589. DOI: 10.1109/TSE.2025.3537337.

[44] A. Cross *et al.*, "OpenQASM 3: A Broader and Deeper Quantum Assembly Language," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, Sep. 2022. DOI: 10.1145/3505636.

[45] J. Liu and H. Zhou, "Systematic Approaches for Precise and Approximate Quantum State Runtime Assertion," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 179–193. DOI: 10.1109/HPCA51647.2021.00025.

[46] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The Oracle Problem in Software Testing: A Survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. DOI: 10.1109/TSE.2014.2372785.

[47] S. R. Shahamiri, W. M. N. W. Kadir, and S. Z. Mohd-Hashim, "A Comparative Study on Automated Software Test Oracle Methods," in *2009 Fourth International Conference on Software Engineering Advances*, 2009, pp. 140–145. DOI: 10.1109/ICSEA.2009.29.

[48] R. A. Oliveira, U. Kanewala, and P. A. Nardi, "Chapter Three - Automated Test Oracles: State of the Art, Taxonomies, and Trends," in ser. Advances in Computers, A. Memon, Ed., vol. 95, Elsevier, 2014, pp. 113–199. DOI: 10.1016/B978-0-12-800160-8.00003-6.

[49] M. Pezzè and C. Zhang, "Chapter One - Automated Test Oracles: A Survey," in ser. Advances in Computers, A. Memon, Ed., vol. 95, Elsevier, 2014, pp. 1–48. DOI: 10.1016/B978-0-12-800160-8.00001-2.

[50] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[51] S. Shamshiri, J. Campos, G. Fraser, and P. McMinn, "Disposable Testing: Avoiding Maintenance of Generated Unit Tests by Throwing Them Away," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 2017, pp. 207–209. DOI: 10.1109/ICSE-C.2017.100.

[52] M. A. Gulzar, Y. Zhu, and X. Han, "Perception and Practices of Differential Testing," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 71–80. DOI: 10.1109/ICSE-SEIP.2019.00016.

[53] R. B. Evans and A. Savoia, "Differential testing: a new approach to change detection," in *The 6th Joint Meeting on European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering: Companion Papers*, ser. ESEC-FSE companion '07, Dubrovnik, Croatia: Association for Computing Machinery, 2007, 549–552, ISBN: 9781595938121. DOI: 10.1145/1295014.1295038.

[54] G. Casella and R. L. Berger, *Statistical inference*, 2nd ed. Duxbury, 2001.

[55] P. M. Kroonenberg and A. Verbeek, "The tale of cochran's rule: My contingency table has so many expected values smaller than 5, what am i to do?" *The American Statistician*, vol. 72, no. 2, pp. 175–183, 2018. DOI: 10.1080/00031305.2017.1286260.

[56] W. G. Cochran, "Some methods for strengthening the common χ^2 tests," *Biometrics*, vol. 10, no. 4, pp. 417–451, 1954. [Online]. Available: http://www.jstor.org/stable/3001616.

[57] A. Javadi-Abhari *et al.*, *Quantum computing with Qiskit*, 2024. DOI: 10.48550/arXiv.2405.08810. arXiv: 2405.08810.

[58] Q. Wang. "How to check a given unitary evolution is correct in a real quantum computer in Qiskit? - Quantum Computing Stack Exchange." (Jan. 2023), [Online]. Available: https://quantumcomputing.stackexchange.com/questions/29784/how-to-check-a-given-unitary-evolution-is-correct-in-a-real-quantum-computer-in (visited on 07/17/2025).

[59] K. Patel, J. Hayes, and I. Markov, "Fault testing for reversible circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 23, no. 8, pp. 1220–1230, 2004. DOI: 10.1109/TCAD.2004.831576.

[60] M. Zamani, M. B. Tahoori, and K. Chakrabarty, "Ping-pong test: Compact test vector generation for reversible circuits," in *2012 IEEE 30th VLSI Test Symposium (VTS)*, 2012, pp. 164–169. DOI: 10.1109/VTS.2012.6231097.

[61] B. Mondal, C. Bandyopadhyay, and H. Rahaman, "A testing scheme for mixed-control based reversible circuits," in *2016 Sixth International Symposium on Embedded Computing and System Design (ISED)*, 2016, pp. 96–100. DOI: 10.1109/ISED.2016.7977062.

[62] K. Patel, J. Hayes, and I. Markov, "Fault testing for reversible circuits," in *Proceedings. 21st VLSI Test Symposium, 2003.*, 2003, pp. 410–416. DOI: 10.1109/VTEST.2003.1197682.

[63] Y. Syamala, A. V. N. Tilak, and K. Srilakshmi, "Testing of reversible combinational circuits," in *Advances in Communication, Network, and Computing*, V. V. Das and J. Stephen, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 46–53, ISBN: 978-3-642-35615-5.

[64] J. Mondal and D. K. Das, "A new online testing technique for reversible circuits," *IET Quantum Communication*, vol. 3, no. 1, pp. 50–59, 2022. DOI: https://doi.org/10.1049/qtc2.12035.

[65] C. Developers, *Cirq*, version v1.4.0, May 2024. DOI: 10.5281/zenodo.11398048.

[66] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B AkashNarayanan, A. Asadi, *et al.*, *Pennylane: Automatic differentiation of hybrid quantum-classical computations*, 2018. DOI: 10.48550/arXiv.1811.04968. arXiv: 1811.04968.

[67] A. Miranskyy, M. Khan, and U. C. Mendes, "Comparing algorithms for loading classical datasets into quantum memory," in *IEEE International Conference on Quantum Computing and Engineering*, IEEE, 2024, pp. 234–238. DOI: 10.1109/QCE60285.2024.10284.

[68] D. Fortunato, J. CAMPOS, and R. ABREU, "Mutation testing of quantum programs: A case study with qiskit," *IEEE Transactions on Quantum Engineering*, vol. 3, pp. 1–17, 2022. DOI: 10.1109/TQE.2022.3195061.

[69] E. Mendiluze, S. Ali, P. Arcaini, and T. Yue, "Muskit: A Mutation Analysis Tool for Quantum Software Testing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 1266–1270. DOI: 10.1109/ASE51524.2021.9678563.

[70] A. Arcuri and L. Briand, "A Hitchhiker's Guide to Statistical Tests for Assessing Randomized Algorithms in Software Engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014, ISSN: 1099-1689. DOI: 10.1002/stvr.1486.

[71] R. Yin, *Case Study Research: Design and Methods* (Applied Social Research Methods). SAGE Publications, 2009, ISBN: 9781412960991.

[72] C. Wohlin, P. Runeson, M. Höst, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering* (Computer Science). Springer Berlin Heidelberg, 2012, ISBN: 9783642290442.

[73] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd international conference on software engineering*, 2011, pp. 1–10.

[74] D. I. Sjøberg and G. R. Bergersen, "Construct validity in software engineering," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1374–1396, 2022.

[75] R. Takagi, S. Endo, S. Minagawa, and M. Gu, "Fundamental limits of quantum error mitigation," *npj Quantum Information*, vol. 8, no. 1, p. 114, 2022.

[76] Z. Cai, R. Babbush, S. C. Benjamin, S. Endo, W. J. Huggins, Y. Li, J. R. McClean, and T. E. O'Brien, "Quantum error mitigation," *Reviews of Modern Physics*, vol. 95, no. 4, p. 045 005, 2023.

[77] R. Mandelbaum, J. Gambetta, J. Chow, T. Mittal, T. J. Yoder, A. Cross, and M. Steffen. "IBM lays out clear path to fault-tolerant quantum computing." (Jun. 2025), [Online]. Available: https://www.ibm.com/quantum/blog/large-scale-ftqc.

[78] E. T. Campbell, B. M. Terhal, and C. Vuillot, "Roads towards fault-tolerant universal quantum computation," *Nature*, vol. 549, no. 7671, pp. 172–179, 2017. DOI: 10.1038/nature23460.

[79] R. J. Wieringa and M. Daneva, "Six strategies for generalizing software engineering theories," *Science of computer programming*, vol. 101, pp. 136–152, 2015. DOI: 10.1016/J.SCICO.2014.11.013.

[80] D. Fortunato, J. Campos, and R. Abreu, "Mutation testing of quantum programs written in qiskit," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, 358–359, ISBN: 9781450392235. DOI: 10.1145/3510454.3528649.

[81] D. Fortunato, J. Campos, and R. Abreu, "QMutPy: a mutation testing tool for Quantum algorithms and applications in Qiskit," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2022, Virtual, South Korea: Association for Computing Machinery, 2022, 797–800, ISBN: 9781450393799. DOI: 10.1145/3533767.3543296.

[82] J. Campos and A. Souto, "QBugs: A Collection of Reproducible Bugs in Quantum Algorithms and a Supporting Infrastructure to Enable Controlled Quantum Software Testing and Debugging Experiments," in *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*, 2021, pp. 28–32. DOI: 10.1109/Q-SE52541.2021.00013.

[83] P. Zhao, Z. Miao, S. Lan, and J. Zhao, "Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs," *Journal of Systems and Software*, vol. 205, p. 111 805, 2023, ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111805.

[84] D. Coppersmith, *An approximate fourier transform useful in quantum factoring*, 2002. DOI: 10.48550/arXiv.quant-ph/0201067. arXiv: quant-ph/0201067.

[85] "QFT — Qiskit v.1.3.1." (2024), [Online]. Available: https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.QFT (visited on 12/26/2024).

[86] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1996, pp. 212–219.

[87] M. Paltenghi and M. Pradel, "MorphQ: Metamorphic testing of the qiskit quantum computing platform," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, 2023, pp. 2413–2424.

[88] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.